

Metaheurísticas de Mejora Iterativa (Búsqueda Local) *Iterative Improvement (Local Search)*

- **Enfriamiento Simulado** (*Simulated Annealing*)
 - **Búsqueda Tabú** (*Tabu Search*).
 - **Búsqueda por Haz** (*Beam Search*).

Bibliografía:

- Monografía: Metaheurísticas. Inteligencia Artificial, Vol. 1, No. 40 (Quinto trimestre), (2003).
- Inteligencia Artificial. Técnicas, métodos y aplicaciones. Palma, Marín. McGraw Hill (2008). (CAP. 9)
- **Artículos** en Poliformat. Detalles de metaheurísticas, Estudios comparativos

Tema-3 Articulos

 [Busqueda Tabu.pdf](#)

 [Enfriamiento simulado.pdf](#)

 [Learning to rank using SA.pdf](#)

 [Simulated Annealing_Theory_with_Applications.pdf](#)

 [Simulated Annealing.pdf](#)

Búsqueda de Soluciones

META-HEURÍSTICOS

HEURÍSTICOS
heurística: $f(n)$

Best-First
Search

BÚSQUEDA
LOCAL

BÚSQUEDA
GLOBAL
(sistemática)

Escalada

Haz

Voraz

A*

IDA*

SMA*

Poca memoria
No Completos

Completos
Mucha memoria

Mejora Metaheurística
Haz, Tabú,
Enfriamiento Simulado,

Híbridos: Constructivos + Mejora Local
GRASP

Evolutivos

Algoritmos Genéticos
(Búsqueda dispersa, Alg. Meméticos)

Inteligencia Social

Hormigas, Abejas, Partículas

Los metodos de busqueda local suelen caer en max locales.

*Hace falta poder escapar de max locales
⇒ Metaheursiticas.*

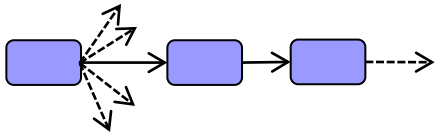
Métodos Metaheurísticos. Tipos Básicos.

Construcción Solución Optimizada
(Pasos iterativos: +Elementos Solución)

Métodos Constructivos:
Algoritmo A, $A^* \Rightarrow$ **GRASP**

Búsqueda Local / Trayectoria

Mejora de una Solución
Requiere Solución Inicial
(Pasos iterativos: Mejora Solución)
Búsqueda en un espacio de Soluciones



Mejora Conjunto Soluciones
(Pasos: Evoluciones Cooperativas)

Mejora Iterativa de UNA SOLUCIÓN

Enfriamiento Simulado
Búsqueda Tabú

Mejora Iterativa CONJUNTO SOLUCIONES
(no colaborativa)

Búsqueda en Haz

Control Centralizado:
Evoluciones poblacionales
(Pobl. Individuos ~ Soluciones)

Alg. Genéticos
Alg. Meméticos
Búsqueda Dispersa

Control Autónomo:
Inteligencia de Enjambre
Enjambre de Individuos (Soluciones)

Alg. Hormigas
Colonia de Abejas,
Enjambre de Partículas...

Generación & mejora

Metaheurísticas de Mejora (búsqueda local)

Idea base: Mejora iterativa de la solución mediante **búsqueda local (y limitada)** en el entorno , pasando de una solución a otra vecina mejor (algoritmos de escalada)

BASE: Escalada (hill-climbing)

- Se parte de una solución inicial.
- Se obtienen sus “vecinos”, en una búsqueda en la vecindad.
- Se sustituye la solución por un mejor vecino, si cumple el criterio de aceptación.
- Se repiten los pasos anteriores mientras se satisfaga el criterio de continuación.
- El proceso se acabaría cuando no hay mejora posible en el conjunto de soluciones vecinas (valles, llanuras, ciclos, etc.).

Esquema general:

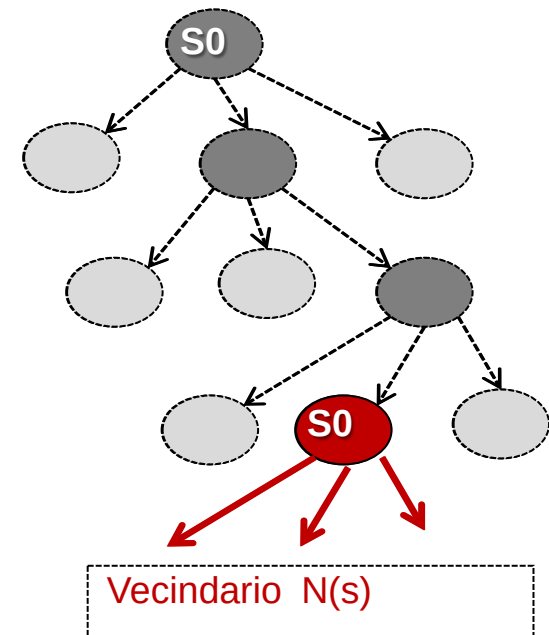
Generar una solución inicial $s_0 \in S$.

Repetir:

 Seleccionar $s \in N(s_0)$, tal que $f(s) < f(s_0)$

 Reemplazar s_0 por s

hasta que $\forall s \in N(s_0), f(s) \geq f(s_0)$.

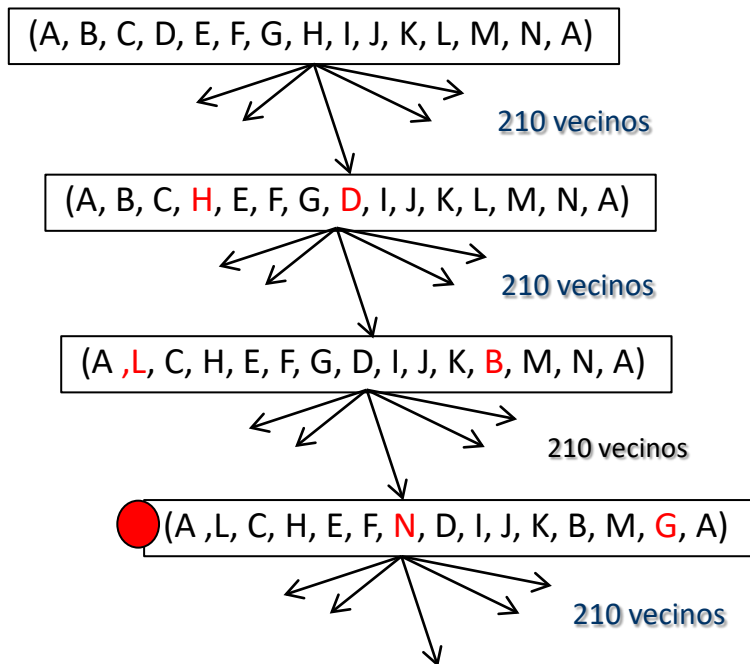


Ejemplo: Viajante (15 ciudades)

Inicial: (A, B, C, D, E, F, G, H, I, J, K, L, M, N, A)

Entorno: Intercambiar ?x por ?y

(15 * 14 = 210 vecinos,
de 15!=10³⁶ soluciones)



Esquema general:

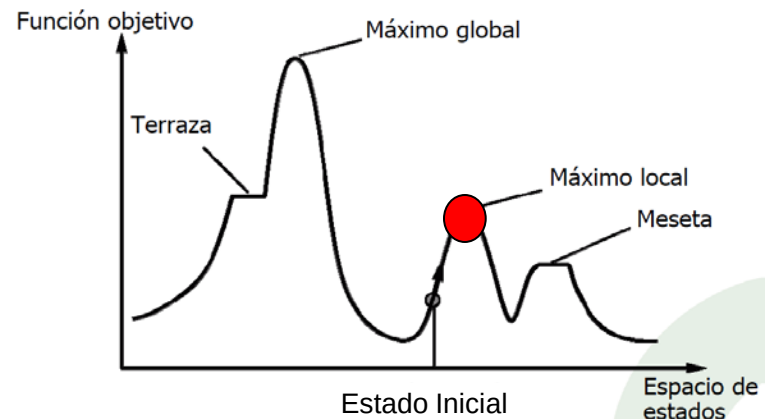
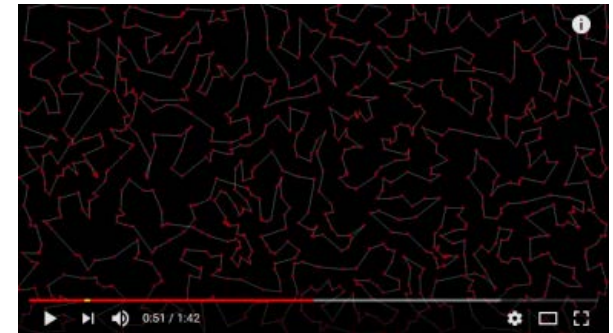
Generar una solución inicial $s_0 \in S$.

Repetir:

→ Seleccionar $s \in N(s_0) / f(s) < f(s_0)$

Reemplazar s_0 por s

hasta que $\forall s \in N(s_0), f(s) \geq f(s_0),$.



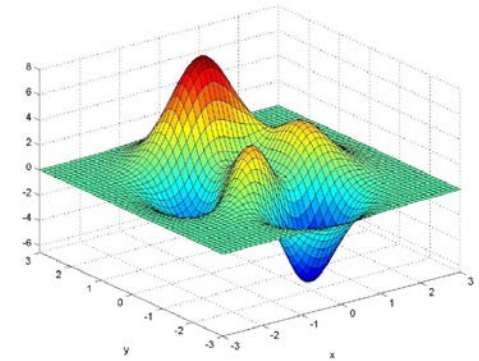
¿Y si ninguna solución vecina mejora?

En resumen, la Escalada.....

- Recorre un espacio de soluciones transformando iterativamente una solución de partida.
- Requiere:
 - Una solución inicial $s_0 \in S$, y una función de evaluación de las soluciones $f(s)$.
 - Definir estructura de entorno N , $\forall s \in S$, define $N(s) \subseteq S$.
 - *Debe garantizar:*
 - Factibilidad, Baja cardinalidad $|N(s)|$, Oportunidad de mejora, Facilidad de evaluación, etc.

Ventajas:

- Poca memoria.
- A menudo encuentran soluciones razonables, en espacios de estados en los que la búsqueda global es inabordable.



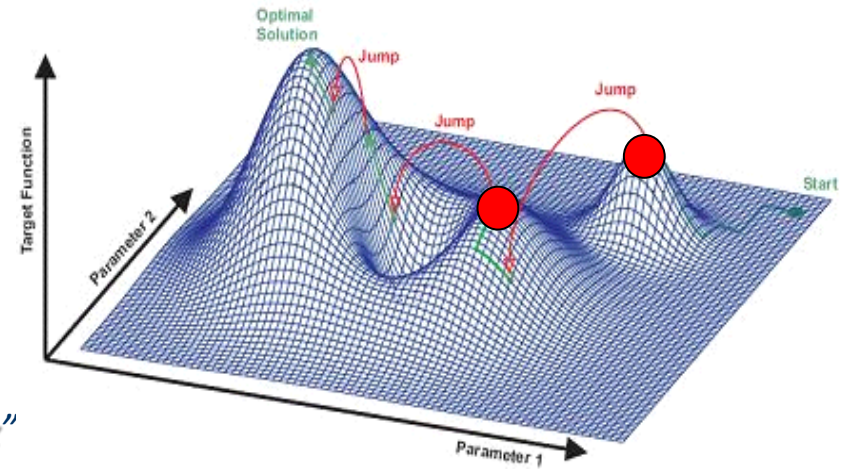
Inconvenientes:

- La búsqueda puede acabar en máximo/mínimo local.
- Se obtienen soluciones localmente óptimas, pero pueden estar lejos del óptimo global.
- Bucles (debido a no almacenar en memoria).



¿Posibles Soluciones?: (escapar máximo local)

- Salto a otra zona de búsqueda previa (*backjumping*).
- Avanzo al siguiente nivel, aunque sea peor.
- Re-arranque (*arranque múltiple*)



Re-arranque (“si no te sale a la primera, inténtalo otra vez”)

- Se reinicia la búsqueda con una nueva solución cuando ésta se ha estancado.
Se repite varias veces, partiendo cada vez de un estado inicial distinto
- Si la probabilidad de éxito de una búsqueda individual es p , entonces el número esperado de reinicios es $1/p$. Se queda con la mejor solución de todos los multiarranques.

No es propiamente una metaheurística

Ventaja: Escapa de valles

Desventaja: Perdida de la información de búsqueda en cada reinicio.

¿Alternativas?

Algoritmo: Escalada con Reinicio

$S_{\text{actual}} \leftarrow S_0$ (*Solución_Inicial*), $S_{\text{mejor}} \leftarrow S_{\text{actual}}$

mientras no_condición_parada **hacer**

$S_{\text{nueva}} \leftarrow \text{Mejor } \{S \in \text{Vecinos}(S_{\text{actual}})\}$

si $f(S_{\text{nueva}}) > f(S_{\text{actual}})$

entonces $S_{\text{mejor}} \leftarrow S_{\text{nueva}}$

$S_{\text{actual}} \leftarrow S_{\text{nueva}}$

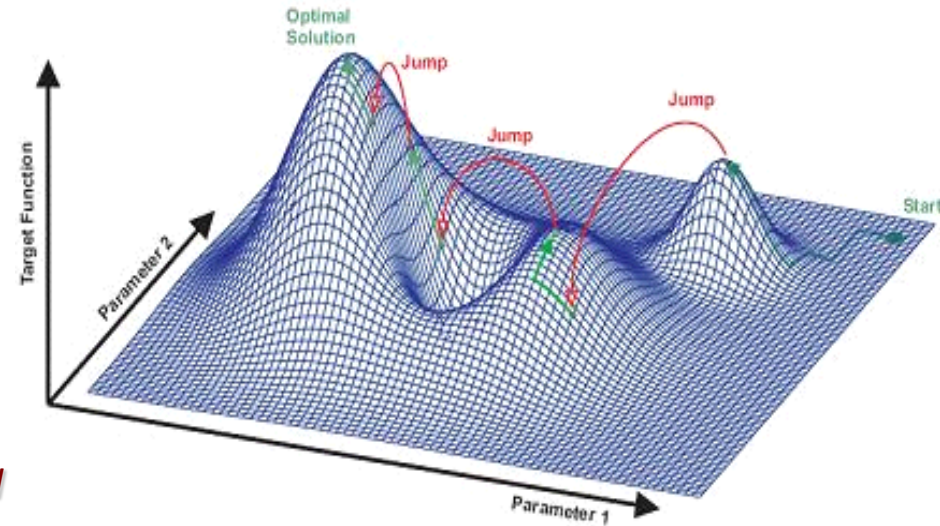
sino $S_{\text{actual}} \leftarrow \text{Generar}(S_{\text{inicial}})$

fin

Fin ; devuelve S_{mejor}

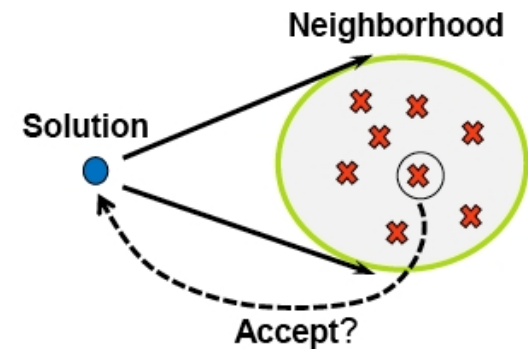
Alternativas:

- a) Aceptar (a veces) movimientos que empeoran la calidad de la solución actual.
- b) Modificar los entornos $N(s)$ durante la búsqueda
- c) Ampliar la búsqueda local



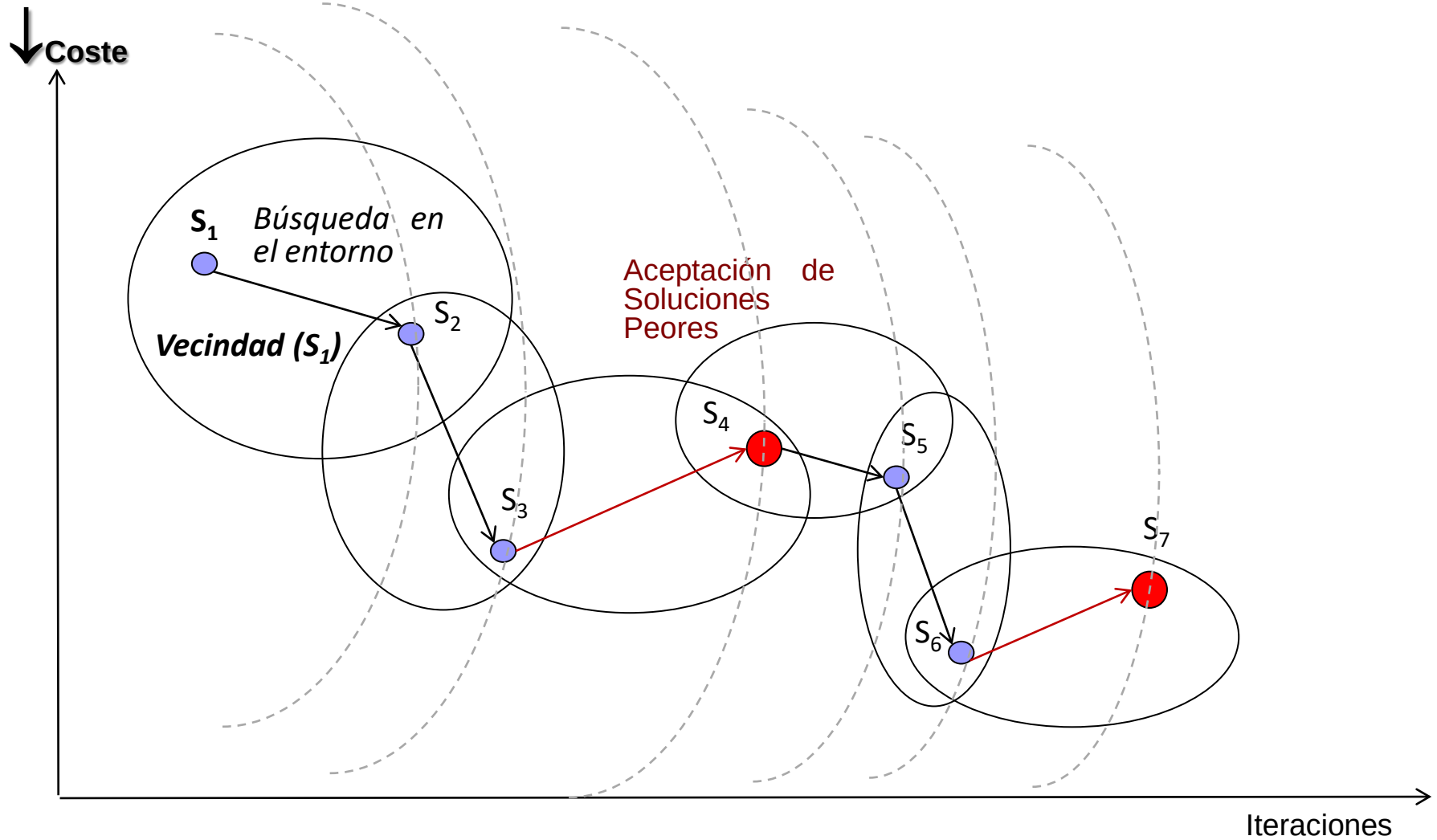
Metaheurística: *estrategia de búsqueda global* para **escapar de óptimos locales** de baja calidad

- a) Metaheurísticas de **búsqueda no monótona**:
Permiten movimientos que no sean de mejora.
Enfriamiento Simulado (*simulated annealing*)
- b) Metaheurísticas de **entorno (vecindad) variable**:
Modifican la estructura de entornos locales que se aplica.
Búsqueda Tabú (*búsqueda con memoria*), Vecindades Variables
- c) Metaheurísticas de **búsqueda por haz limitado**:
Búsqueda en haz.



Metaheurísticas de Mejora:

Búsqueda en un Espacio de Soluciones / Búsqueda Local /
Metaheurísticas de Trayectoria / Búsqueda en entornos



Enfriamiento simulado (*Simulated Annealing, Recocido/Temple Simulado*).

Kirkpatrick, Gelatt, Vecchi "Optimization by Simulated Annealing". Science 220 (4598):671–680

Annealing: "Proceso de enfriar lentamente un material en estado líquido hasta un estado cristalino de mínima energía. Si la temperatura se reduce de manera suficientemente lenta, el material obtendrá su estado de más baja energía (ordenación perfecta)"

Aplicaciones

SA from
2022

Ideas Básicas:

1. Realizar una **exploración más amplia al principio**, ya que la solución final es relativamente insensible a los estados iniciales y se disminuye la probabilidad de caer en mínimos locales, valle o cresta en zonas iniciales de la búsqueda.
2. **Admitir**, con cierta probabilidad, estados sucesores en los que **$f(x)$ empeore**, a fin de poder salir de óptimos locales. Esta probabilidad disminuye conforme avanza la búsqueda.
3. **Concentrar** (guiar) más la búsqueda en las **etapas finales**.

Esquema Escalada:

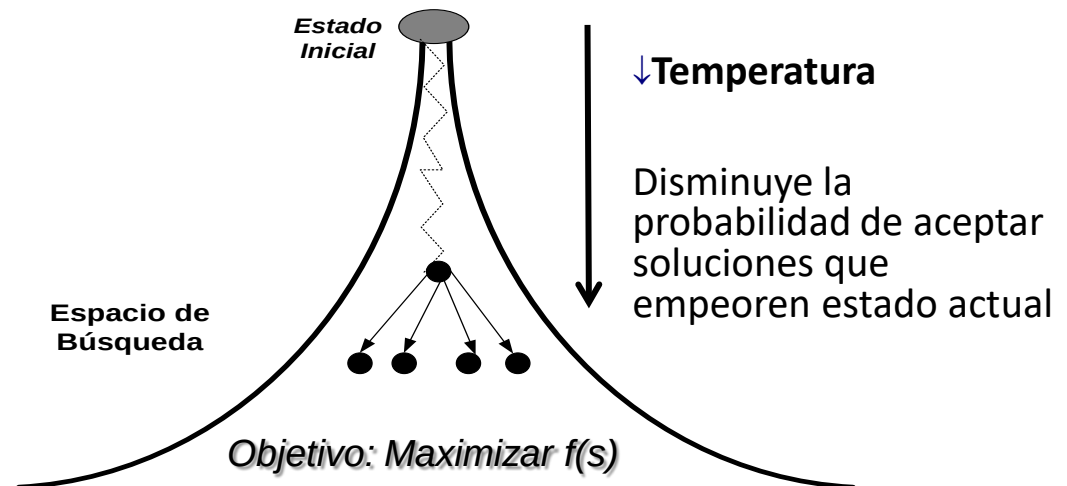
Generar una solución inicial $s_0 \in S$.

Repetir:

Seleccionar $s \in N(s_0)$ tal que **$f(s) > f(s_0)$**

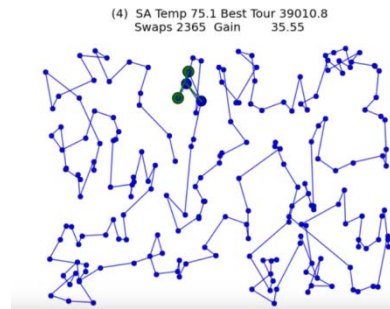
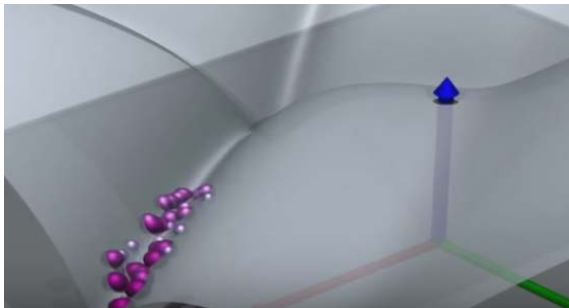
Reemplazar s_0 por s

hasta que



Método:

- Elección aleatoria de un vecino/sucesor del estado actual ($\cong$ *Escalada estocástica*)
 - a) Si es **mejor**, se elije incondicionalmente.
 - b) Si es **peor**, Se acepta el estado con una probabilidad que depende de la **temperatura (T)** y el incremento de **energía (ΔE)**, asimilado al decremento del valor de $f(n)$
- La temperatura (T) **va bajando conforme avanza la búsqueda** (a cada iteración)
 - Por ello, la **probabilidad** de aceptar un estado que empeore el actual:
va bajando a medida que avanza la búsqueda....
.... además de cuánto empeora la función objetivo
- Hacia el final de la búsqueda, solo se admiten soluciones que mejoren o igualen la actual.



Esquema Enfriamiento Simulado (maximizar $f(x)$)

En cada iteración, en vez de elegir el mejor movimiento sucesor, se elige un **sucesor aleatorio**:

- a) Si el movimiento mejora la situación ($\Delta f \geq 0$), entonces es aceptado
- b) Si no mejora la situación ($\Delta f < 0$), se acepta el movimiento con una probabilidad menor a 1:

Típicamente, **Probabilidad** = $e^{\Delta f/T}$ ($e \cong 2.718$)

Esta probabilidad se decrementa:

- Con el empeoramiento de la evaluación Δf (ya que $\Delta f < 0$)
- Con la temperatura (T), que va bajando conforme avanza la búsqueda:

$T_{i+1} = \alpha(\text{Iteración}, T_i)$, tal que T decrece al aumentar la iteración

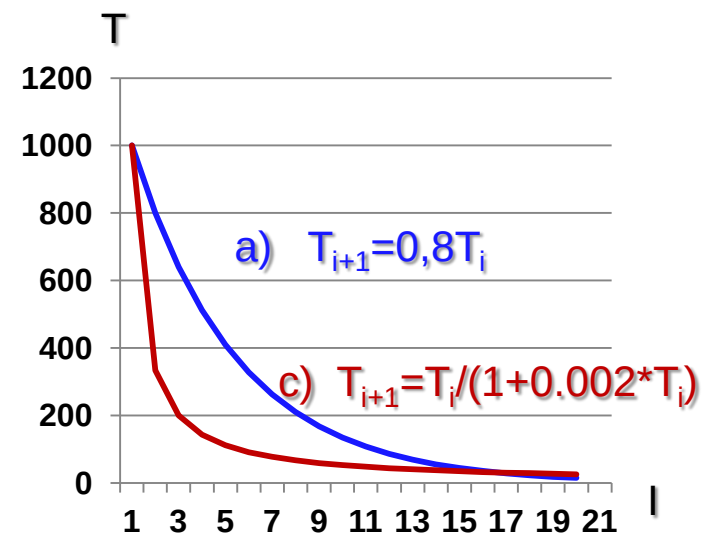
Inicialmente, a valores altos de T , se acepta casi cualquier estado.
Al bajar la temperatura, se van dejando de aceptar estados no buenos

El decremento α se puede definir de varias formas, ejemplos:

- a) $\alpha(i, T) = T_{\text{INICIAL}} - i \cdot k$ (lineal)
- b) $\alpha(i, T) = k \cdot T_i$, con $0 < k < 1$, típicamente $\{0,8, 0,99\}$
- c) $\alpha(i, T) = T_i / (1 + kT_i)$, $k > 0$ y con valor muy pequeño.

Si la variación de T es lenta y T_{INICIAL} es alto, la probabilidad de llegar a una buena solución es mayor, pero la búsqueda es más costosa.

El ajuste (T_{INICIAL} y k) dependen del problema y hay que realizarlo de forma experimental.



Procedimiento de Enfriamiento simulado

$S_{\text{ACTUAL}} := S_{\text{INICIAL}}; \quad S_{\text{MEJOR}} := S_{\text{ACTUAL}}; \quad T := T_{\text{INICIAL}}; \quad l=0; \quad ;\text{Parámetros iniciales}$

Mientras “Existan sucesores de S_{ACTUAL} ” y “criterio_terminación=falso” **hacer**

$S_{\text{NUEVO}} \leftarrow \text{Operador-aleatorio}(S_{\text{ACTUAL}})$

; Movimiento aleatorio, no el mejor!

$\Delta f \leftarrow f(S_{\text{NUEVO}}) - f(S_{\text{ACTUAL}})$

; ¿Mejora o Empeora?

Si $\Delta f > 0$ entonces *; Mejora la solución: se acepta el movimiento*

$S_{\text{ACTUAL}} \leftarrow S_{\text{NUEVO}}$

Si S_{NUEVO} mejor que S_{MEJOR} entonces $S_{\text{MEJOR}} := S_{\text{NUEVO}}$

Si_no

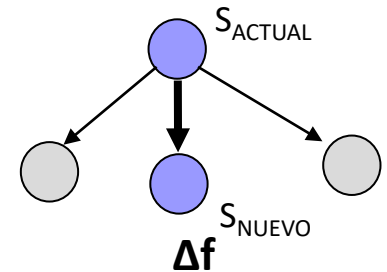
$S_{\text{ACTUAL}} \leftarrow S_{\text{NUEVO}}$ con probabilidad $e^{\Delta f/T}$ *;Podría no actualizarse S_{ACTUAL}*

$l=l+1;$

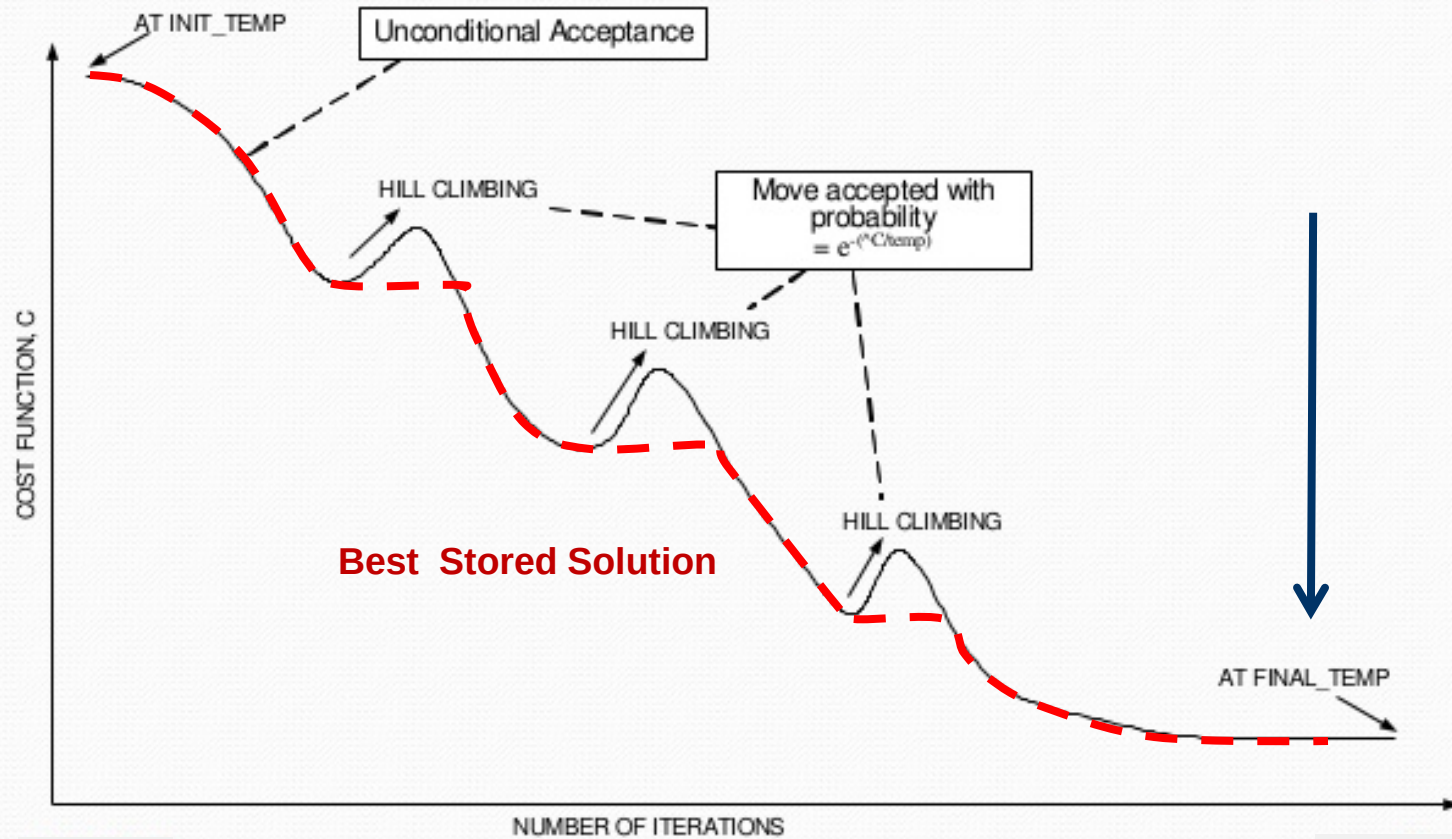
$T = \alpha(l, T)$ *; Actualizar T acorde planificación del enfriamiento, por ejemplo, $T=T/(1+kT)$, siendo $0 < k < 1$*

finMientras

Devolver S_{MEJOR}

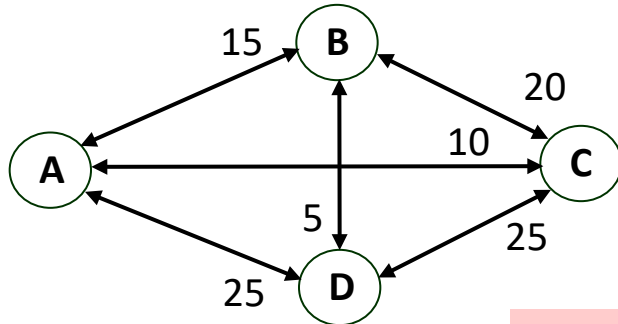


Convergence of simulated annealing



Ejemplo.1: Enfriamiento Simulado

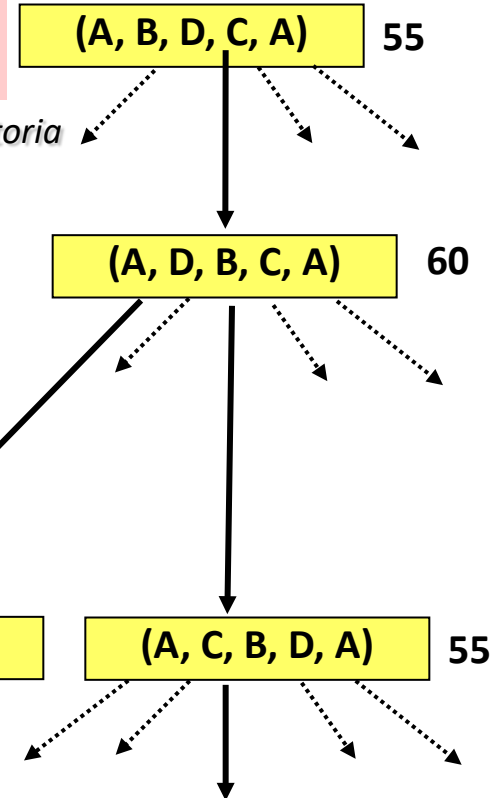
Problema del Viajante



$S_{\text{INICIAL}} = (A, B, D, C, A); S_{\text{ACTUAL}} := S_{\text{INICIAL}};$
 $S_{\text{MEJOR}} := S_{\text{ACTUAL}}; T := 1000; l=0; k=0.01$
 $\alpha(i, T) = T/(1 + 0.01T)$

Solución Inicial
 $l=0, T=1000$

Elección Operador aleatoria



$\Delta f(S_{\text{NUEVO}}) = -5$
 Probabilidad de elección = $e^{\Delta f/T} = e^{-5/1000} = 0,995$
Se acepta. $S_{\text{ACTUAL}} = (A, D, B, C, A)$
 $l=1, T_{i+1} = 1000/(1+0.01*1000) = 90.9$

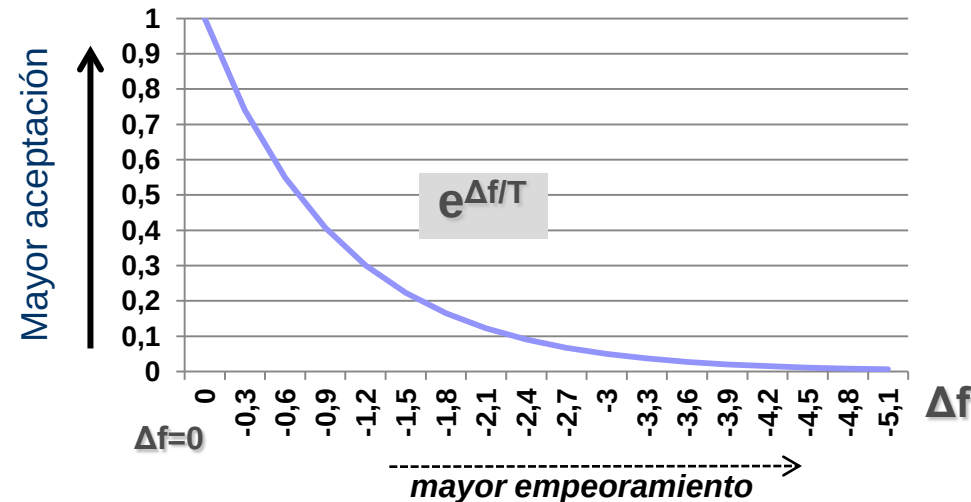
Elección Operador aleatoria

$\Delta f(S_{\text{NUEVO}}) = -25$
 Probabilidad de elección = $e^{\Delta f/T} = e^{-25/90.9} = 0,76$
 No se acepta.
 $l=2, T_{i+1} = 90.9/(1+0.01*90.9) = 47.62$

$\Delta f(S_{\text{NUEVO}}) = 5$
Se acepta ($\Delta f > 0$). $S_{\text{ACTUAL}} = (A, C, B, D, A)$
 $l=3, T_{i+1} = 47.62/(1+0.01*47.62) = 32.26$

Aceptación de un nuevo estado cuando $\Delta f < 0$: Probabilidad $e^{\Delta f/T}$

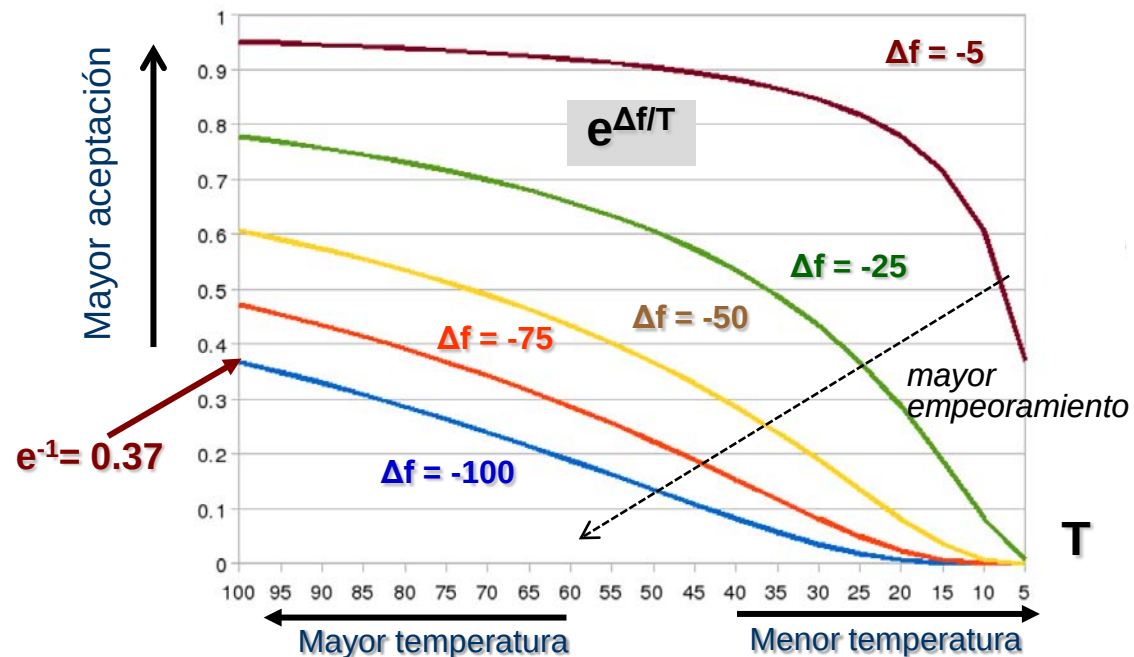
Probabilidad de aceptación $e^{\Delta f/T}$ versus Δf , con $T=1$



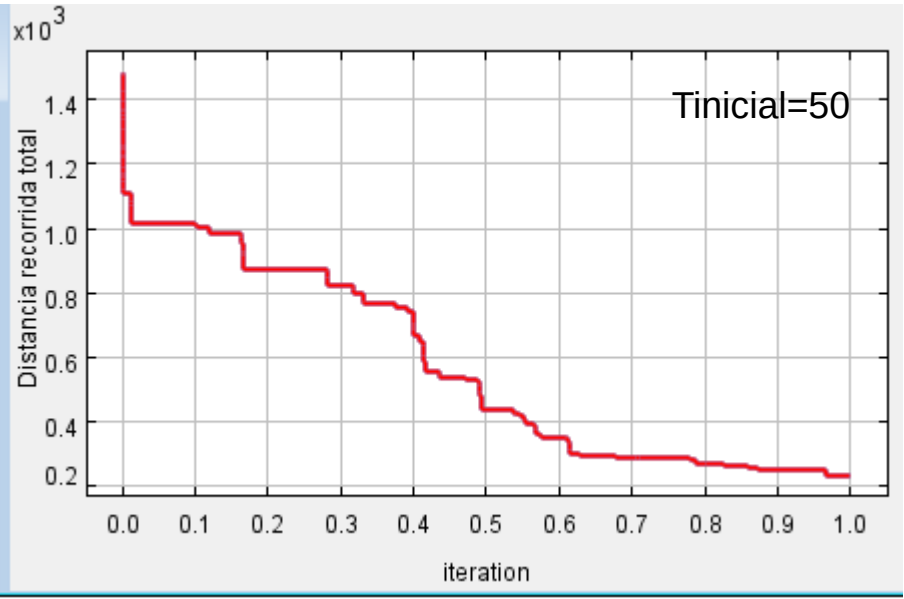
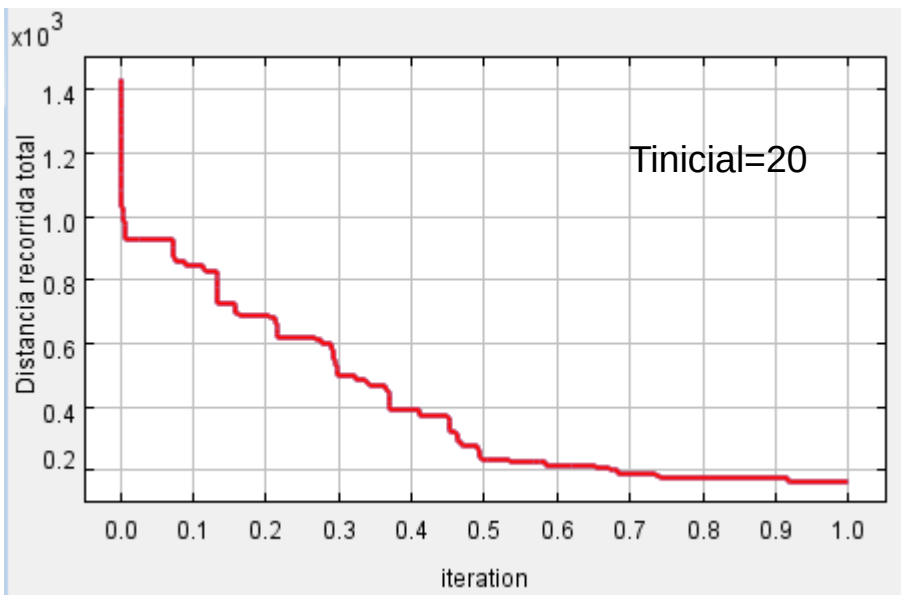
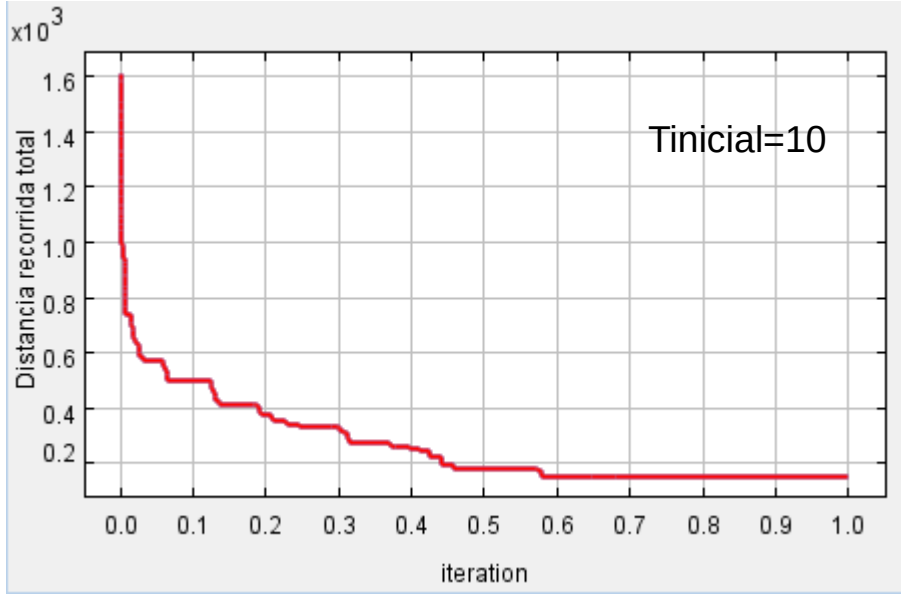
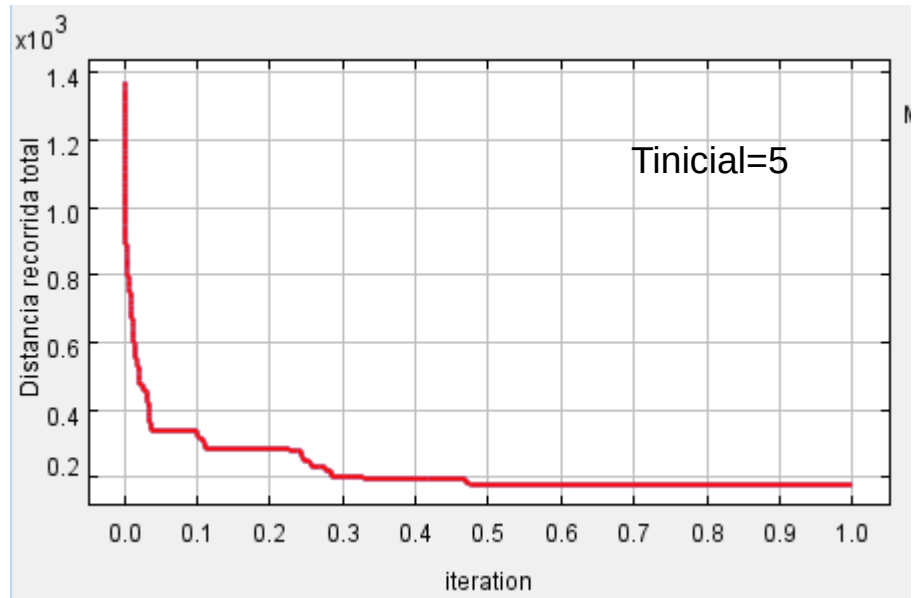
Balance dinámico de la búsqueda controlado a través de la temperatura:

- Alta **diversificación** al inicio (alta temperatura)
- Baja **diversificación** al final (baja temperatura)

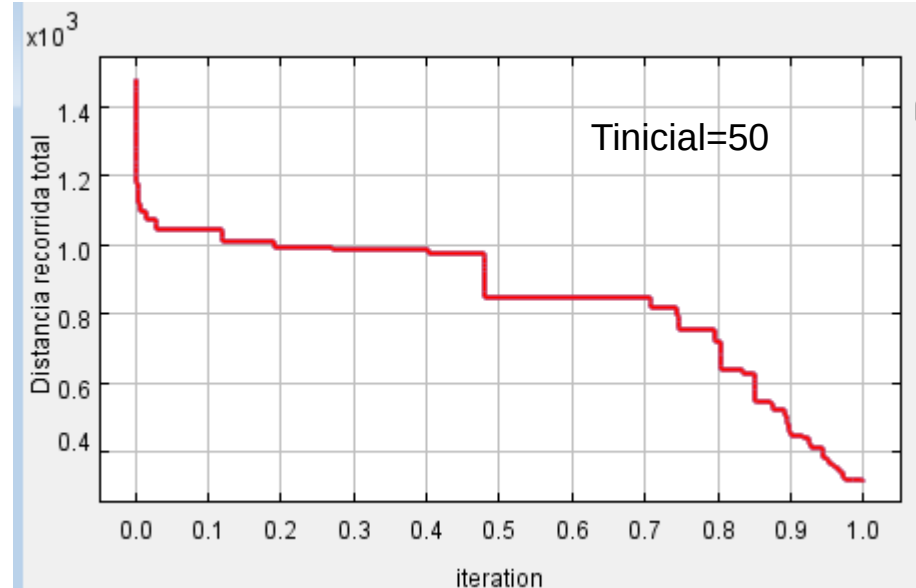
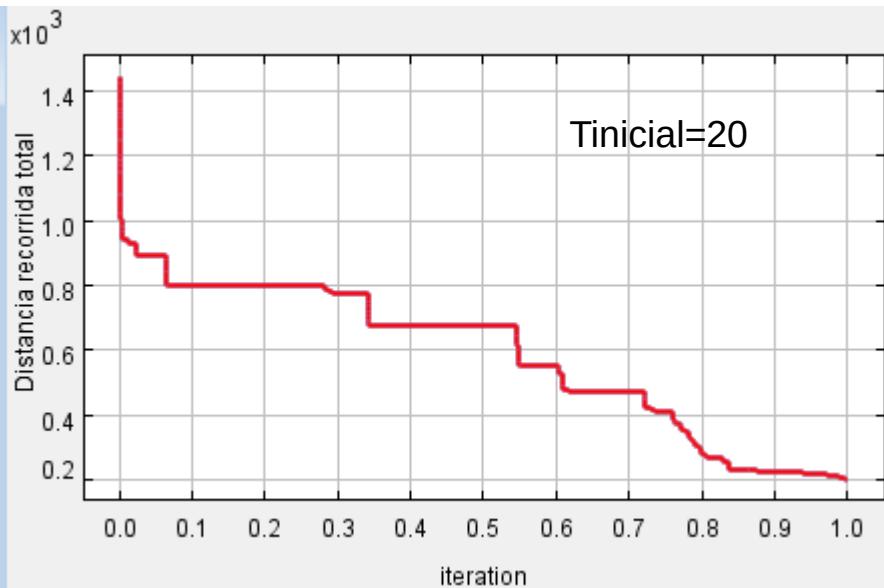
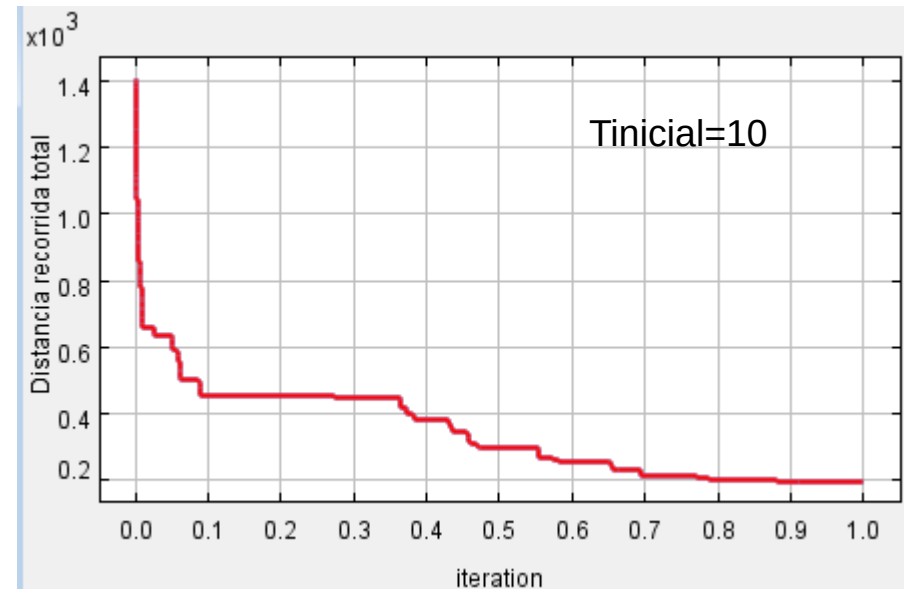
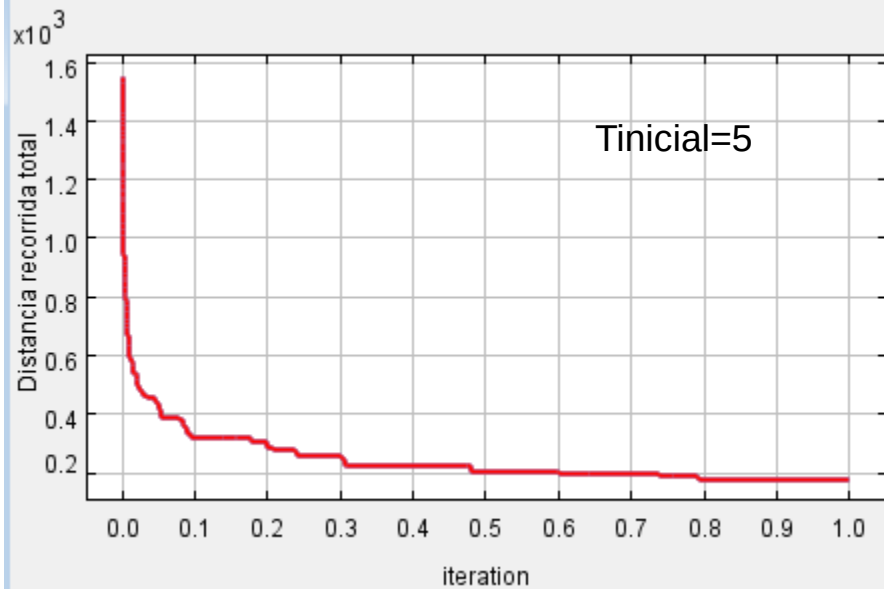
Probabilidad de aceptación $e^{\Delta f/T}$ versus T , (para valores de Δf)



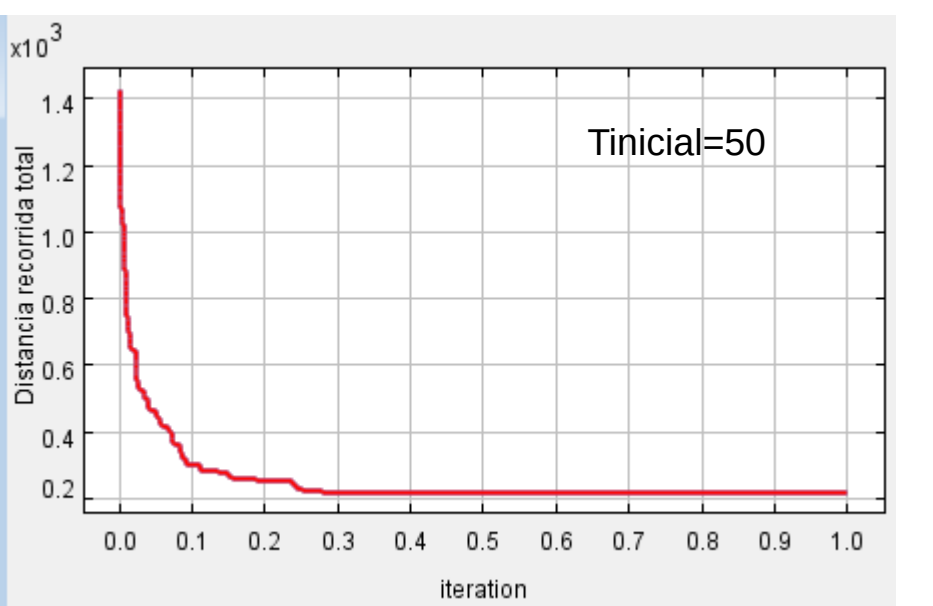
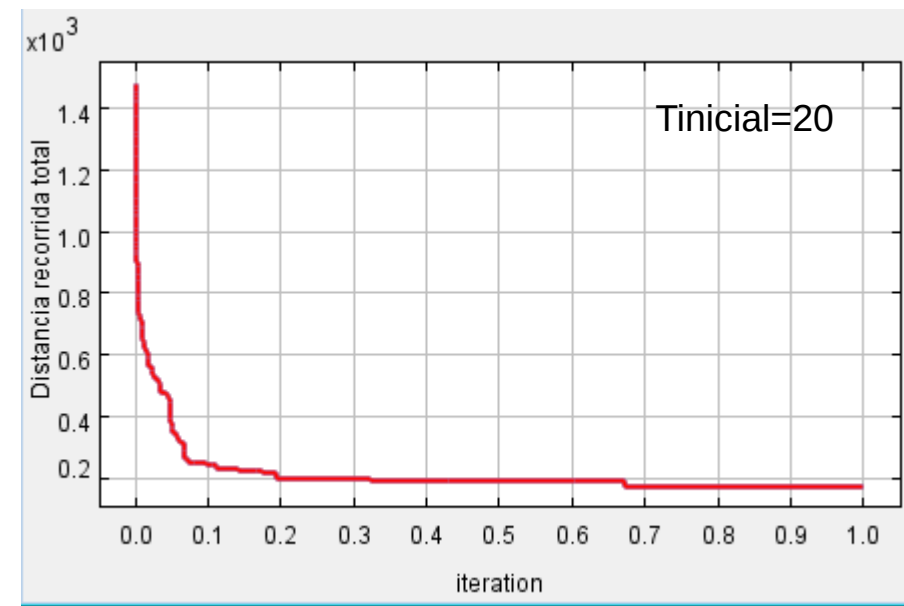
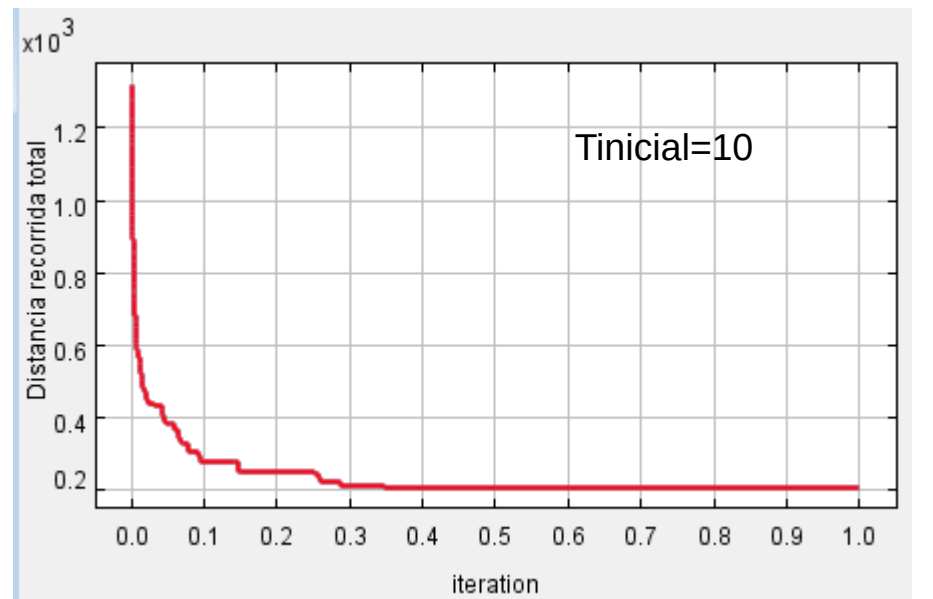
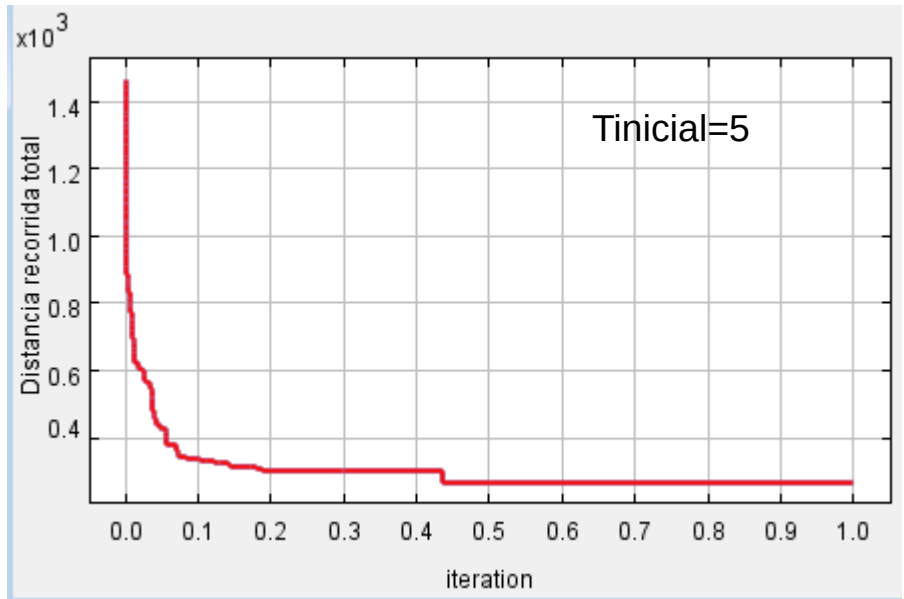
EVALUACIÓN: Convergencia vs. Temperatura Inicial (*iteraciones x 10⁵*). $\alpha(i, T) = k \cdot T_i$



EVALUACIÓN: Convergencia vs. Temperatura Inicial ($\text{iteraciones} \times 10^5$). $\alpha(i, T) = T_{\text{INICIAL}} - i * k$



EVALUACIÓN: Convergencia vs. Temperatura Inicial ($\text{iteraciones} \times 10^5$). $\alpha(i, T) = T_i / (1 + kT_i)$



Aplicaciones

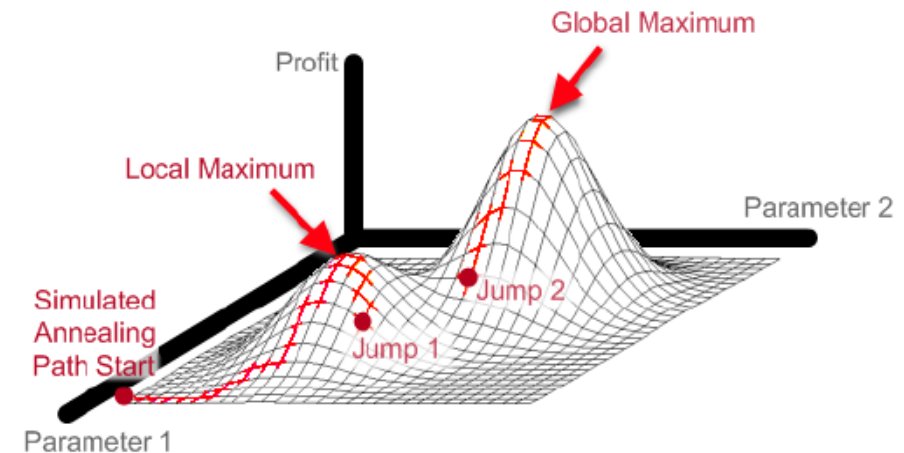
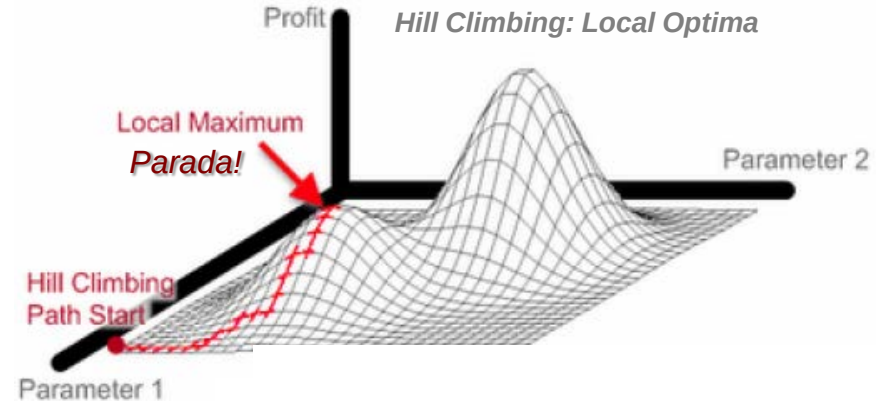
- Indicado para problemas grandes en los que el óptimo está rodeado de **muchos óptimos locales**
- Indicado para problemas en los que encontrar una heurística discriminante es difícil (una elección aleatoria es tan buena como otra cualquiera)

Parametrización:

Temp inicial, función α : experimentación!.

Alta Temp-Inicial, Bajada lenta: Proceso más lento, pero típicamente mejor solución.

Modificaciones: enfriamiento y posterior recalentado (enfriamiento no monótono)



Modificación de la búsqueda local que utiliza distintas vecindades a lo largo de la búsqueda.

Estructura de vecindad: para cada $s \in S$, define el conjunto $N(s) \subseteq S$ de soluciones “vecinas” de s .

$N(s)$ es el conjunto de soluciones a las que se puede llegar desde s aplicando el procedimiento N

El criterio para definir las soluciones vecinas de cada s , $N(s)$, es muy amplio y con alternativas.

$N(s)$ no tienen por qué ser las más cercanas a s : soluciones-vecinas $\neq$ cercanas .

Esquema Búsqueda Local: $N(s)$ es fijo

$s \leftarrow \text{Solucion_Inicial}$

Repeat

$s \leftarrow \text{Mejor}(s, N(s))$

Until no hay mejora posible

Problema: Óptimo Local

- Un óptimo local en una estructura de vecindad, $N_i(s)$, no es necesariamente un óptimo local con otra estructura de vecindad, $N_j(s)$.
- Un óptimo global es óptimo local con todas las posibles estructuras de vecindades

IDEA:

$\Rightarrow$ Definir diversas estructuras de vecindad $N_1, N_2, N_3, \dots, N_k$

Representan como saltos ‘globales’ a otra zona (exploración)

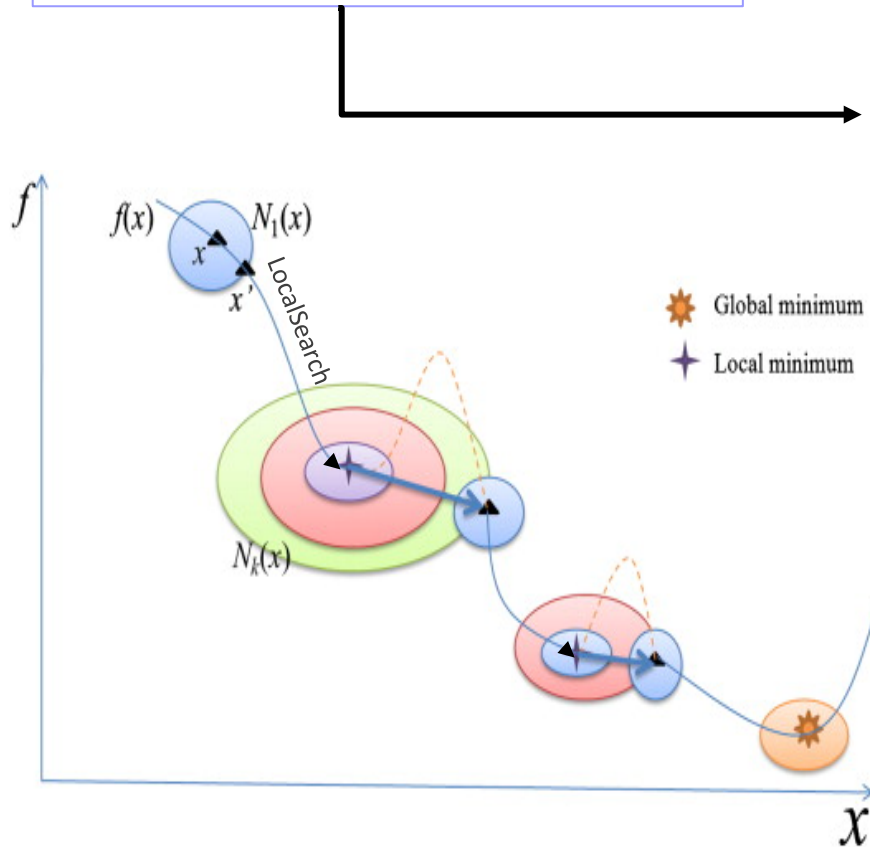
Esquema Búsqueda Local: $N(s)$ es fijo

$s \leftarrow \text{Solucion_Inicial}$

Repeat

$s \leftarrow \text{Mejorar}(s, N(s))$

Until no hay mejora posible



A hybrid approach based on the variable neighborhood search and particle swarm optimization for parallel machine scheduling problems—A case study for solar cell industry Int. J.of Production Economics V.141, I.1, 2013

Se define un conjunto de estructuras de vecindad: $\{N_k\}$, $k = 1, \dots, k_{\max}$

$s \leftarrow \text{SolucionInicial}()$

Mientras no_fin ;recorre la estructuras de vecindad

$k \leftarrow 1$

Mientras $k < k_{\max}$

$s' \leftarrow \text{SeleccionarRandom}(N_k(s))$

$s'' \leftarrow \text{LocalSearch}(s')$;hasta máx-local

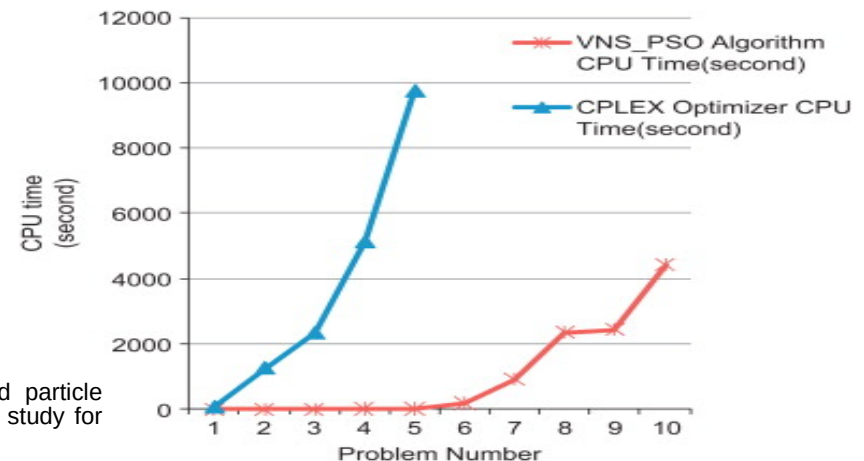
if $f(s'') < f(s)$ then $s \leftarrow s'', k \leftarrow 1$

else $k \leftarrow k+1$

end if

End_mientras

End_mientras



"The general employee scheduling problem: an integration of MS and AI". Glover & McMillan. Computers and Op. Research

Introduce una memoria (lista tabú) para evitar los problemas básicos de la búsqueda local:

➤ **evitar parar en mínimos locales:**

Continúa con la búsqueda local, *aunque sea con movimientos que no mejoren la solución*, para evitar caer en valles.

➤ **entrar en ciclos:**

usa la memoria temporal (lista tabú), que guarda la *historia reciente de la búsqueda*, para evitar estados repetidos.

Conceptos:

- Se parte de una solución, o estado inicial ($x \in X$), que se intenta mejorar.
- Tamaño de la lista tabú es **limitado**. *Suele ser una lista circular (eliminando el elemento más viejo)*
- Lista Tabu (T): movimientos o estados prohibidos. (**Búsqueda de entorno variable**)
 - Evitar caer en soluciones recientemente generadas (*ciclos*).
 - Actúa también como una memoria a corto plazo (olvido estratégico): *valles, óptimos locales*. La búsqueda no se detiene cuando empeora $f(n)$.
- En su versión más simple, la lista tabú es la lista de los recientes estados visitados, pero suele ser un conjunto de estados-tabú no deseados, movimientos o características no deseadas, etc.

Algoritmo Búsqueda Tabú

Se parte de una solución $Sol \in \{Soluciones\}$, inicial

Inicialización (estado inicial Sol):

$Sol^* := Sol$; (mejor solución hasta el momento)
 $k = 0$; (contador de iteración)
 $Tabu := \emptyset$; (lista tabú de estados/movimientos prohibidos: *es una lista circular, limitada*)
 $Suc(Sol)$ (Estructura vecindario o entorno frontera, estados sucesores, accesibles desde Sol)
 $f(s)$ (función coste del estado s)

Mientras no_fin hacer

Identificar $Suc(Sol)$

Si $Suc(Sol) - Tabu = \emptyset$ entonces parar,
si no, $k := k + 1$;

seleccionar Sol, Sol = OPTIMO $\{Suc(Sol) - Tabu\}$

Si $f(Sol) < f(Sol^*)$ entonces $Sol^* := Sol$

Actualizar 'Tabu' (añadir movimiento actual y eliminar elemento viejo, si procede);

Devolver Sol^* (mejor solución encontrada)

*La búsqueda no se detiene
cuando $f(n)$.*

*Posterior
incorporación de
un criterio de
aspiración*

Guarda en Sol^ la mejor, pero
admite soluciones no mejores*

*La lista tabú T de estados prohibidos permite 'saltar' a estados no óptimos
y prohíbe volver a estados previos (o prohibidos)*

Ejemplo-1 (salir de valles, óptimos locales)

- Supongamos que en cada estado solo se consideran dos alternativas (dos movimientos sucesores) y la lista tabu inicialmente vacía y de tamaño 3.

- **Inicialmente estamos en (1).**

Se pueden hacer dos movimientos (hacia 0 y hacia 2).

Se elige el mejor (hacia 2) y se actualiza la lista tabu.

Registramos el movimiento inverso $T=\{\text{mov}(2; 1)\}$ para evitarlo en el futuro.

- **Desde (2) movemos al estado 3** (el mejor y el permitido dada la lista tabú actual).

- **Después movemos al (4).** La lista tabu tiene tres elementos: $\{\text{mov}(4; 3); \text{mov}(3; 2); \text{mov}(2; 1)\}$.

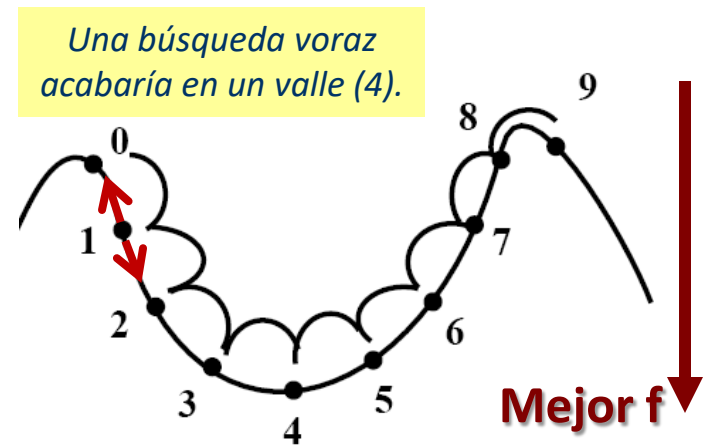
Óptimo local alcanzado!

- **Movemos al punto 5**, es peor en la función objetivo, pero es el mejor (**el único**) de los permitidos por la lista tabú: *moverse a (3) no permitido*.

Se elimina el elemento mas viejo que tiene la lista tabu (por estar llena): $T=\{\text{mov}(5; 4); \text{mov}(4; 3); \text{mov}(3; 2)\}$.

- ...

- **Estando en 8, nos movemos al punto 9**, y salimos del mínimo local. $T=\{\text{mov}(9; 8); \text{mov}(8; 7); \text{mov}(7; 6)\}$.



La búsqueda no se detiene cuando emperora $f(n)$:

- Normalmente la lista tabú se implementa como una *lista circular*, eliminando los primeros añadidos.
- Y típicamente, se va rellenando con los *movimientos inversos* a los realizados (o los *estados ya visitados*).

Ejemplo-2 (salir de ciclos)

Inicialmente estamos en (0) y lista tabú de tamaño 3.
 x^* representa la mejor solución, hasta el momento.

Inicialmente: $T = \emptyset$, $x^*=0$, $Succ(x)=\{1\}$,

Movemos a ($x=1$), $T=\{0\}$, $Succ(1)=\{2\}$, $x^*=1$

Movemos a ($x=2$), $T=\{0, 1\}$, $Succ(2)=\{3, 6\}$, $x^*=2$

Elegimos (3) por ser mejor movimiento

Movemos a ($x=3$), $T=\{0, 1, 2\}$, $Succ(3)=\{4\}$, $x^*=3$

Movemos a (4), $T=\{3, 2, 1\}$, $Succ(4)=\{5\}$, $x^*=4$

Movemos a (5), $T=\{4, 3, 2\}$, $Succ(5)=\{2, 6\}$, $x^*=5$

Mejor movimiento es (2).

Pero (2) no es elegible, porque está en T

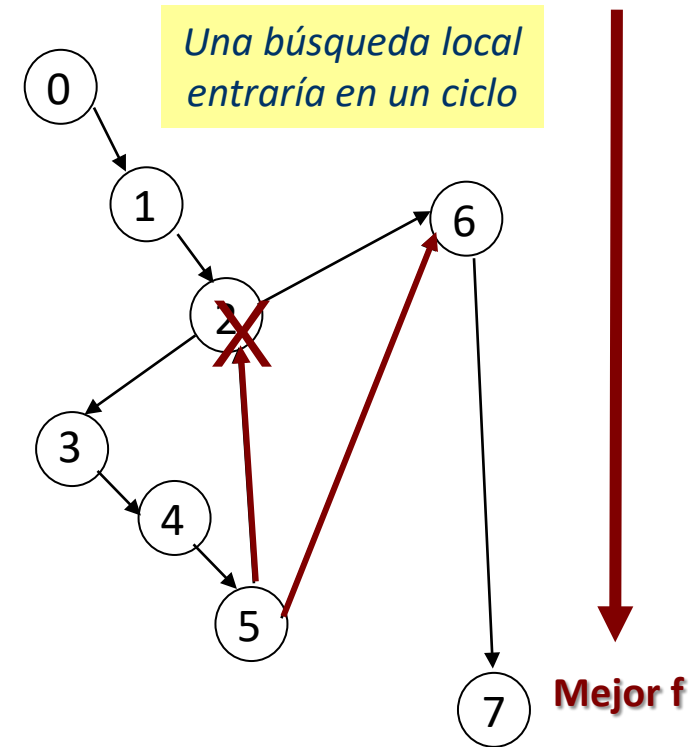
Ciclo Evitado!

Movemos a ($x=6$), $T=\{5, 4, 3\}$, $Succ(6)=\{7\}$, $x^*=5$

Movemos a ($x=7$), $T=\{6, 5, 4\}$, $Succ(7)=\{\}$, $x^*=7$

Para y devuelve $x^*=7$

¿Qué pasaría si la lista tabú tuviera un tamaño igual a 2?



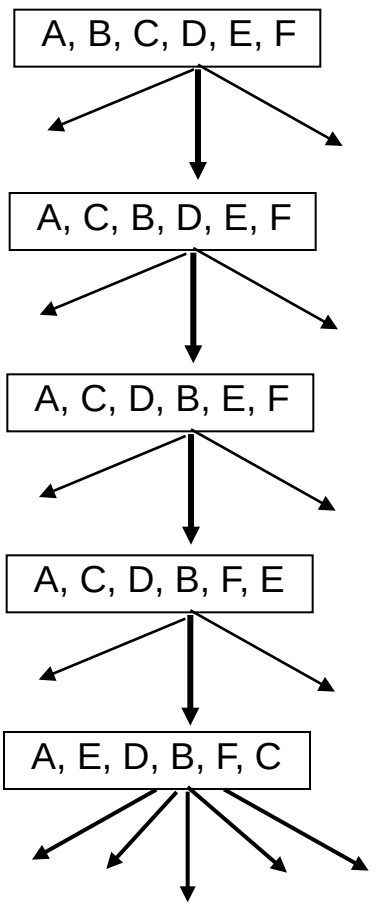
Formas de elaborar la lista Tabú

- **Memoria Explícita:** memoriza en la lista tabú soluciones ya visitadas.
- **Memoria Atributiva:** Memoriza sólo atributos, es decir, características importantes de las soluciones ya visitadas.

Ejemplo 3: Búsqueda Tabú (problema del viajante)

Ciudades: A, B, C, D, E, F
Solución Inicial: A, B, C, D, E, F

La lista tabú representa movimientos prohibidos



Lista Tabú = {}

Permuto (B C)
Lista Tabú = {(B C)}

Permuto (B D)
Lista Tabú = {(B D), (B C)}

Permuto (F E)
Lista Tabú = {(F E), (B D), (B C)}

Permuto (E C)
Lista Tabú = {(E C), (F E), (B D), (B C)}

Búsqueda:

Obtención de un mejor estado sucesor
Sol = OPTIMO {Suc(Sol)-Tabu}

El tamaño de la lista tabú es acotado

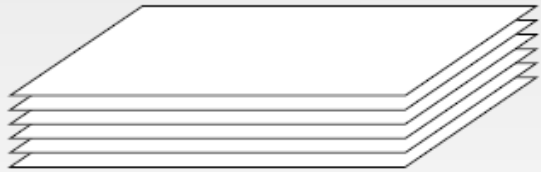
En este estado, no puedo realizar las permutaciones de la lista tabú, es decir, están prohibidos los estados sucesores:

- | | |
|------------------|------------------|
| A, C, D, B, F, E | A, E, B, D, F, C |
| A, F, D, B, E, C | A, E, D, C, F, B |

Ejemplo 4: Búsqueda Tabú

Diseño de un material formado por un número de capas aislantes. El orden según el cual se planifican estas capas determina el valor de aislamiento total del material resultante (Problema de permutaciones, como el TSP):

“Encontrar el orden de las capas que maximiza el valor de aislamiento total del material compuesto”



Solución: Para un problema de 7 capas

2	6	7	3	4	5	1
---	---	---	---	---	---	---

Extraído de: Glover, F., Melián, B. Tabu Search. Inteligencia Artificial, Revista Iberoamericana de Inteligencia Artificial. No.19 (2003),pp. 29-48

- Entorno:** Consideraremos el entorno formado por todos los posibles intercambios de pares. El tamaño del entorno es pues de 21.



- Valor asociado al movimiento:** Todos los intercambios tienen un valor de movimiento asociado, relacionado con el cambio en la función objetivo.

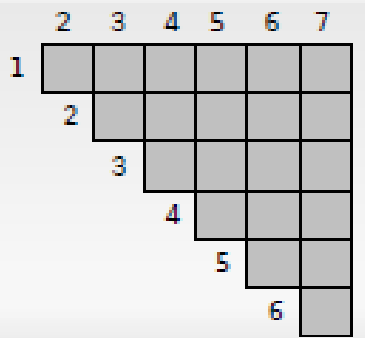
Valor de Intercambio

5, 4	6
7, 4	4
3, 6	2
2, 3	0
4, 1	-1

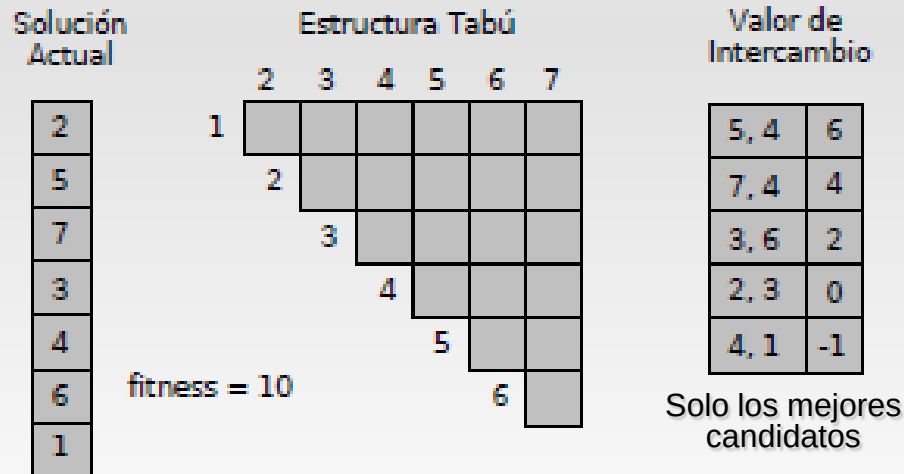
∇f

Los movimientos se almacenan en una estructura tabú:

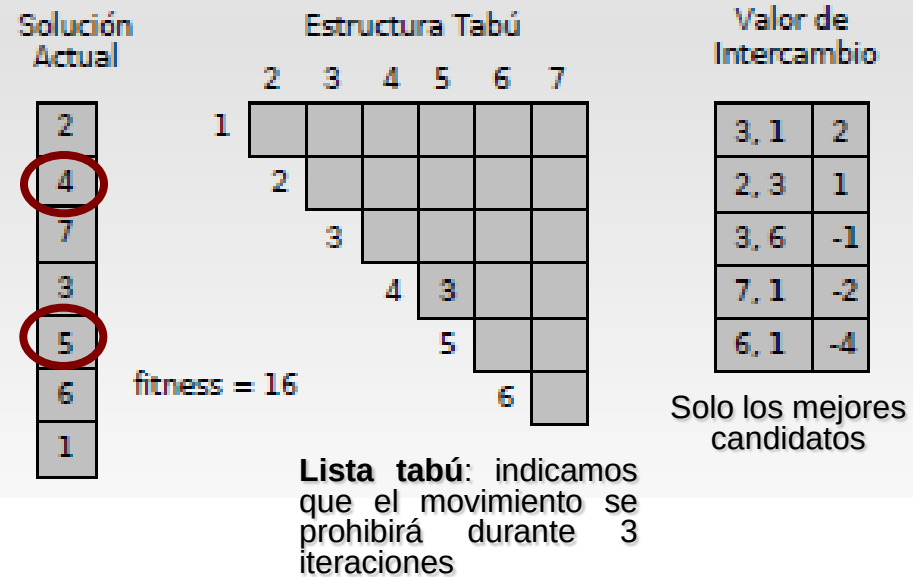
permite guardar los movimientos previos.



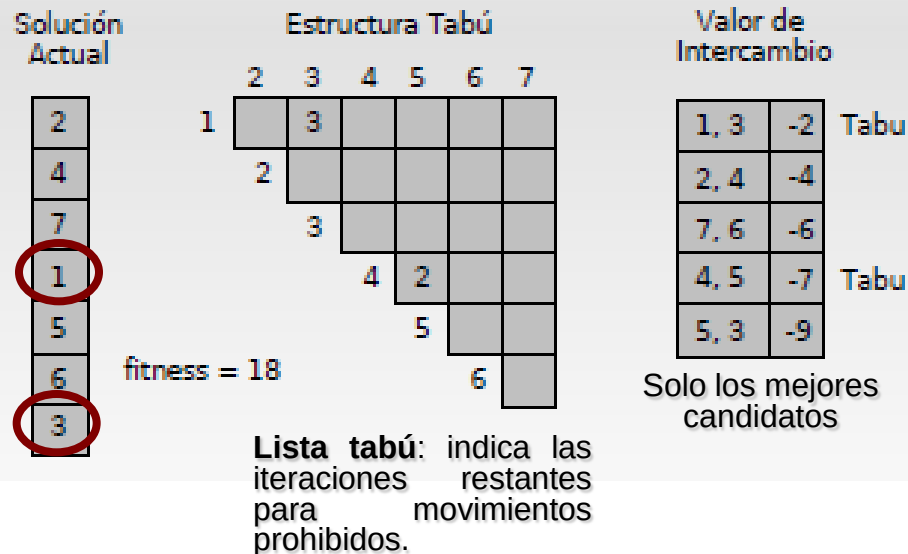
Solución Inicial



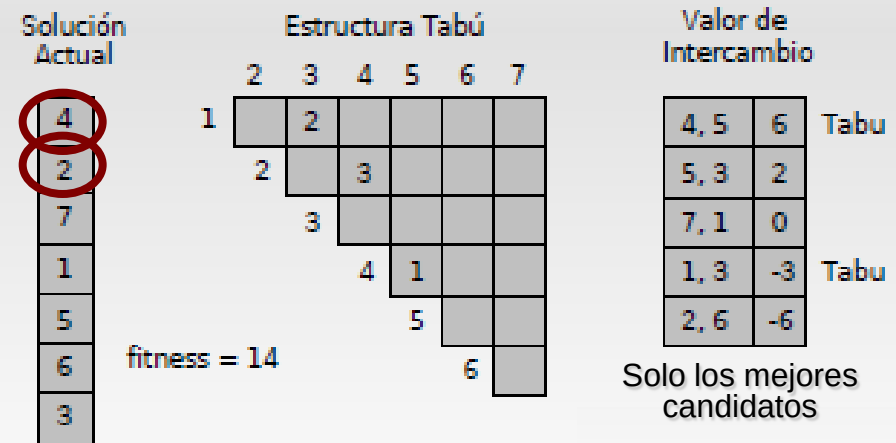
1ª Iteración



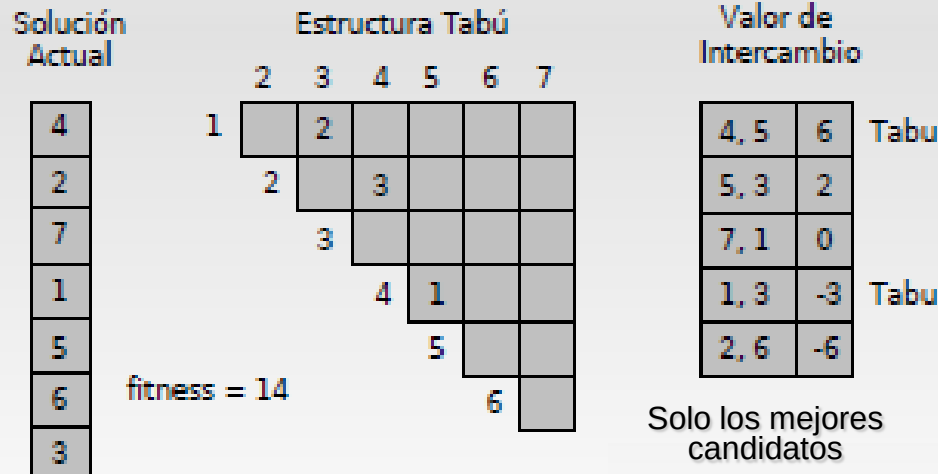
2ª Iteración



3ª Iteración



Tras la 3ª Iteración:

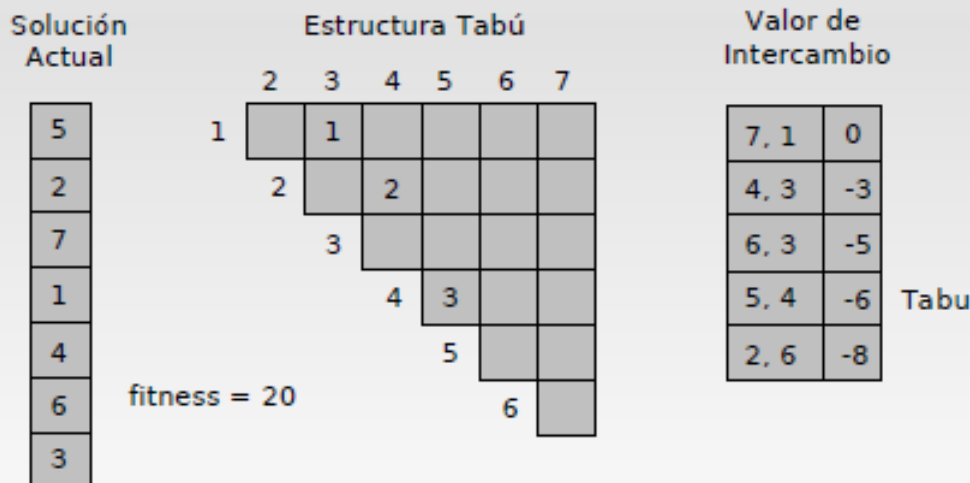


Este intercambio está en la lista tabú (movimiento inverso al primero realizado)

Pero, produce una solución con el *mejor valor* (fitness = 20) que cualquiera previo.

Se hace uso de un **criterio de aspiración**, invalidando la lista tabú, y seleccionando el movimiento.

4ª Iteración:



Ajustes fundamentales de la Búsqueda Tabú:

Tamaño, Criterio aspiración.

Ventajas: Salir de ciclos, Máximos locales

Parámetros Búsqueda Tabú

Tamaño de 'memoria'

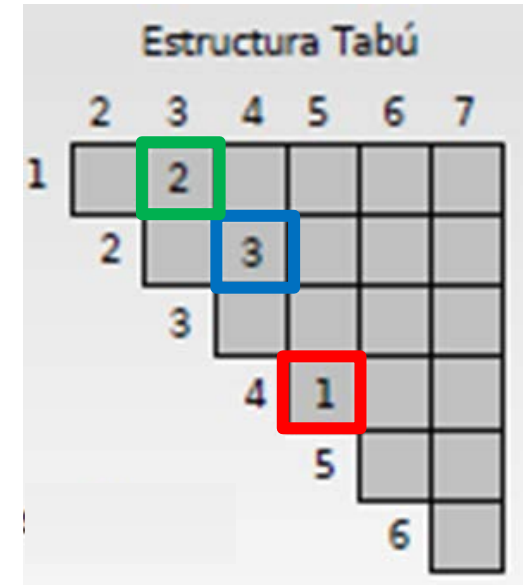
Controlan el proceso de búsqueda, combinando *explotación* (búsqueda local) con *exploración* (búsqueda global, por elementos menos óptimos, pero no en lista tabú).

Balance exploración/explotación:

- **Memoria a corto plazo (↑explotación):** Evitan que la búsqueda se repita (vuelva a visitar la misma solución o la misma región). Los elementos se mantienen poco tiempo (pocas iteraciones) en la lista tabú.
- **Memoria a medio plazo,** para intensificar la búsqueda investigando alguna región potencialmente buena con más detalle.
- **Memoria a largo plazo (↑exploración),** para diversificar la búsqueda investigando regiones no exploradas. Los elementos se mantienen más tiempo en la lista tabú, bloqueando zonas y explorando alternativas.

Criterios de Aspiración

Permiten realizar movimientos prohibidos por la lista tabú bajo determinadas condiciones: alta mejora función objetivo, atributos de la solución, movimiento de alta influencia en la búsqueda, etc.



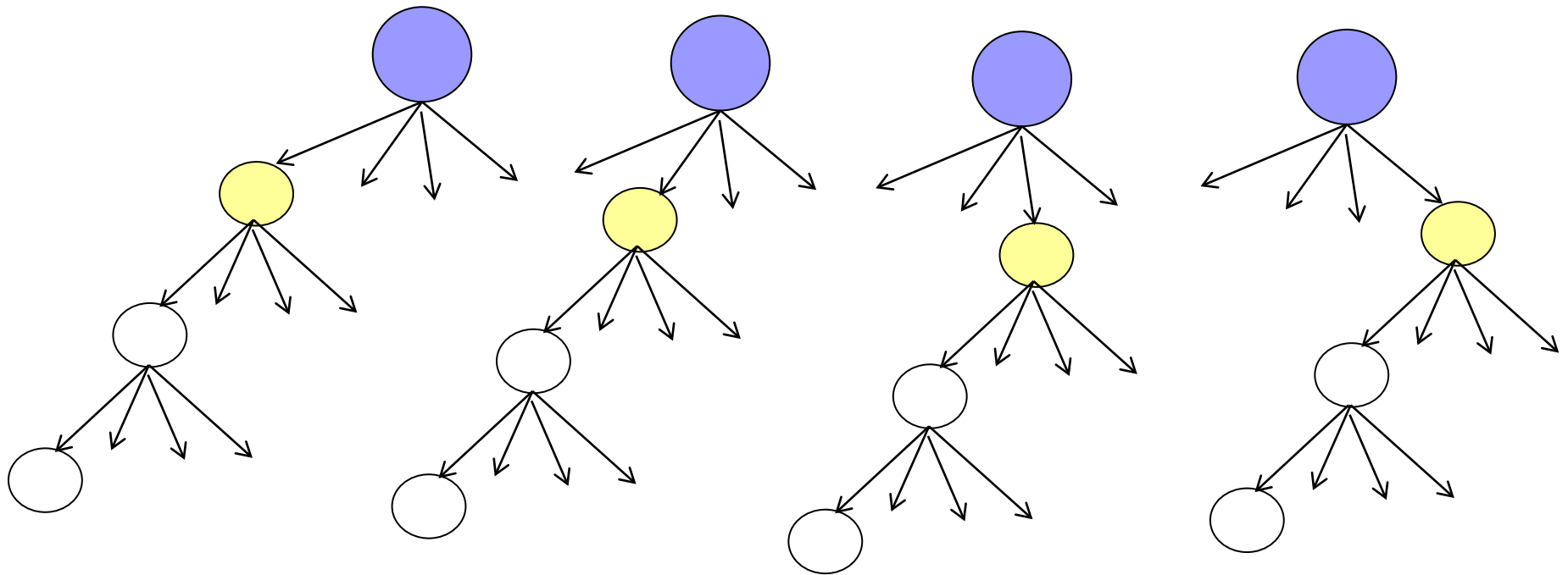
Búsqueda por Haz Local (Beam Search)

"Speech Understanding Systems: A Summary of Results of the Five-Year Research Effort. Dep. Computer Science.", Reddy, D. Raj, , Carnegie Mellon University

IDEA BASE: Escalada con re-arranque (*multiarranque, Random restarting, backjumping*)

- Se reinicia la búsqueda con una nueva solución cuando ésta se ha estancado.

Se repite k veces la búsqueda, partiendo cada vez de un estado inicial distinto, generado aleatoriamente: *"si no te sale a la primera, inténtalo otra vez"*.

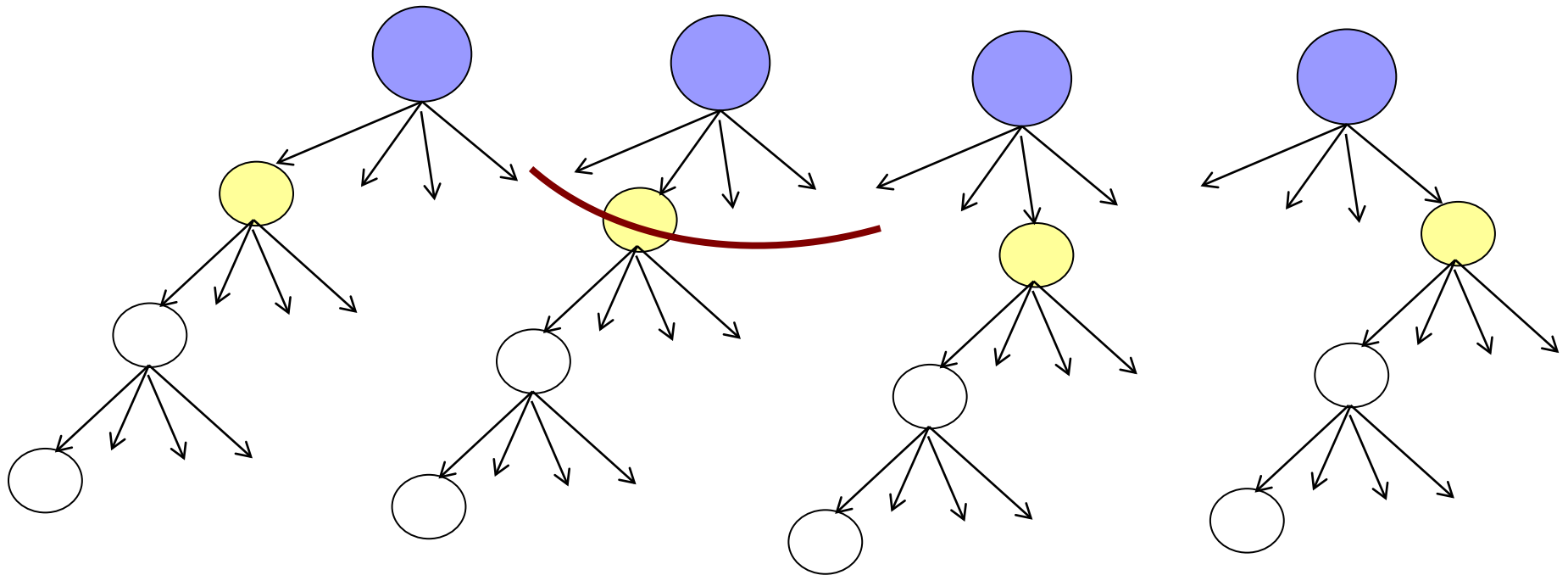


Búsqueda por Haz Local (Beam Search)

IDEA BASE: Escalada con re-arranque (*multiarranque*, *Random restarting*, *backjumping*)

- Se reinicia la búsqueda con una nueva solución cuando ésta se ha estancado.

Se repite k veces la búsqueda, partiendo cada vez de un estado inicial distinto, generado aleatoriamente: *“si no te sale a la primera, inténtalo otra vez”*.

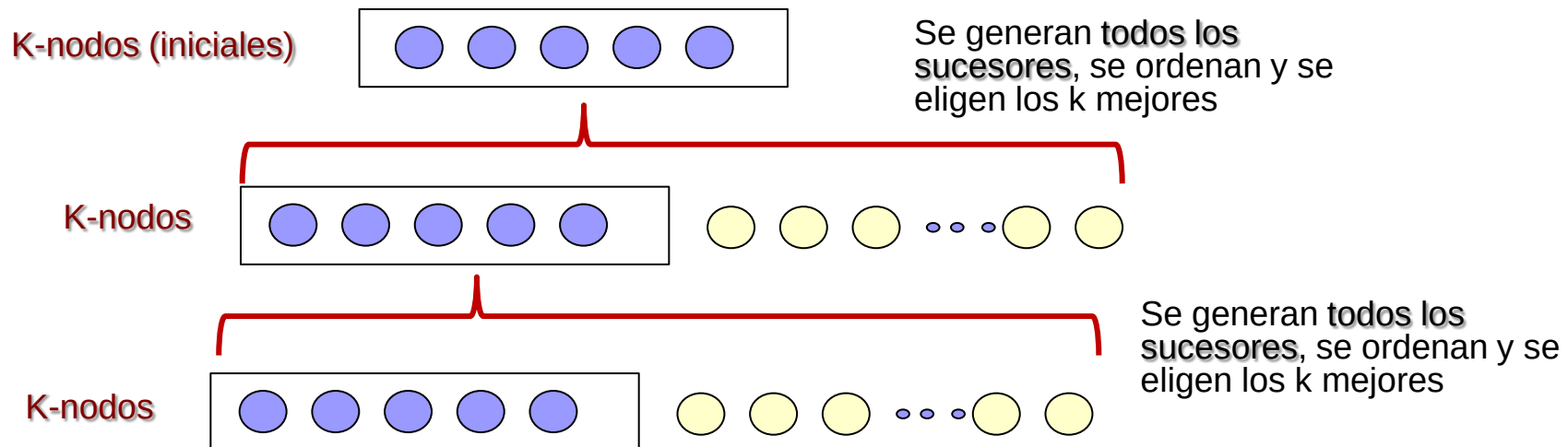


Búsqueda por Haz Local (Beam Search)

Es una Reducción de una Búsqueda Sistemática, limitando la memoria requerida (a costa de la completitud y admisibilidad)

Se mantienen en memoria y se expanden un conjunto limitado de k nodos.

- Comienza con k estados generados aleatoriamente. Si alguno es objetivo (o se cumple la condición de terminación), se detiene la búsqueda
- En cada iteración, se generan todos los sucesores de los k estados y se ordenan según $f(n)$.
 - Si alguno es objetivo o se cumple condición de terminación, se detiene la búsqueda
 - Si no, se seleccionan los k mejores sucesores (haz) y se repite el proceso
- Es diferente a lanzar en paralelo k escaladas! con reinicio aleatorio, pues se generan k -hilos de búsqueda. Si un hilo genera los mejores sucesores, los k hilos de búsqueda seguirán por ese camino.
- La amplitud del haz (k) puede ser fija o variable.

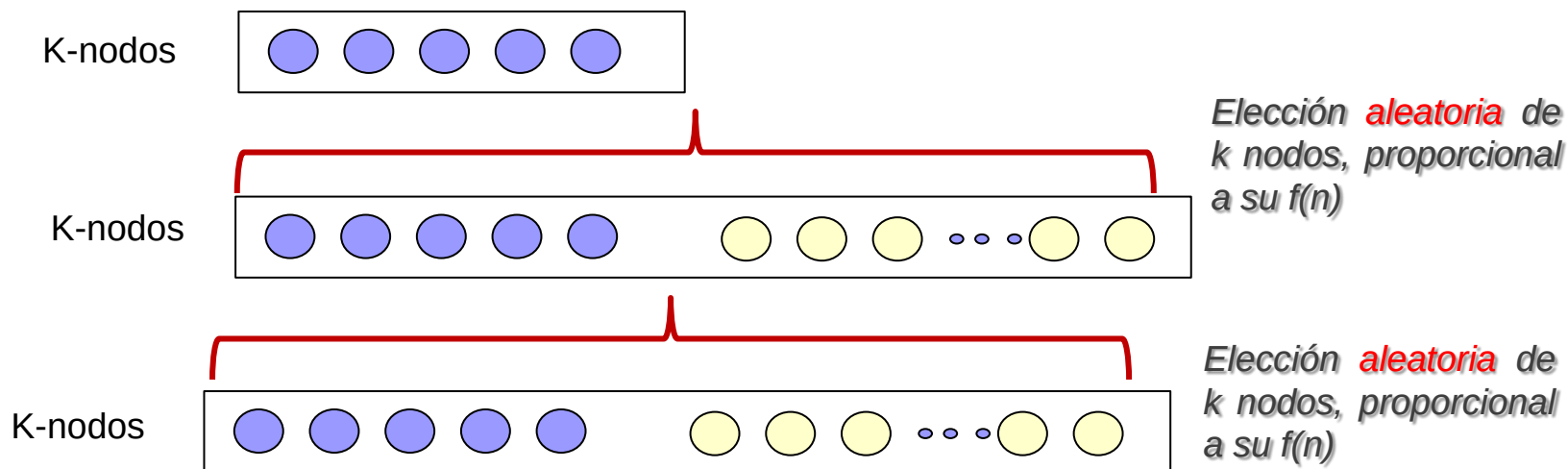


Búsqueda por haz estocástica

- La búsqueda por haz local puede concentrarse rápidamente en pequeñas regiones del espacio de estados y puede ser útil explorar otras zonas aparentemente peores
 - La Búsqueda por haz estocástica trata de combinar la explotación de las zonas mejores con la exploración de las zonas aparentemente peores
- En vez de elegir los k-mejores sucesores, **se eligen k-sucesores con una probabilidad que es función creciente de su valoración:**

Los mejores tienen mayor probabilidad de ser elegidos, aunque no siempre lo serán

- Guarda relación con metaheurísticas basadas en la selección natural (métodos poblacionales).



Resumen: Metaheurísticas de Mejora Iterativa (Búsqueda Local)

- Requieren una solución inicial (factibilidad) y determinar la Estructura de vecindad
- Diversas aplicaciones en problemas complejos:
 - búsqueda de sendas, asignación de recursos, problemas de rutas, configuración, corte de material, etc.

Ajuste Parámetros/ Evaluación:

Tiempo de Cómputo

Vs

Soluciones evaluadas

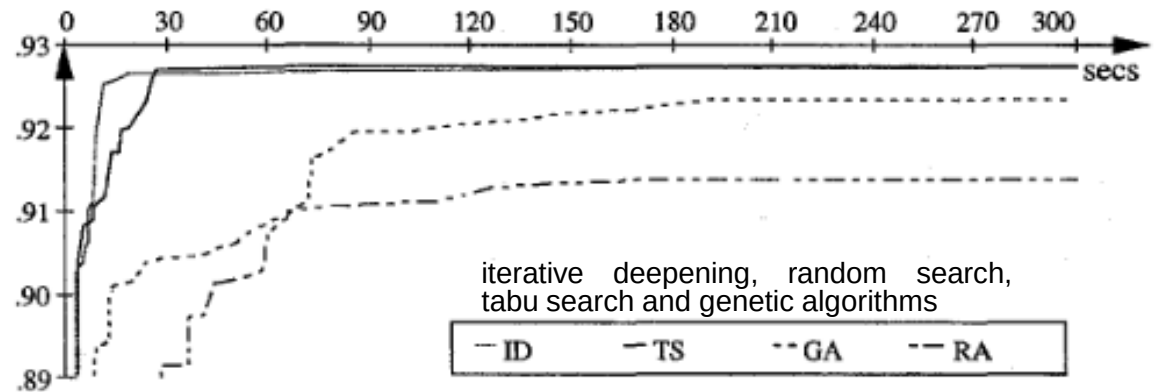
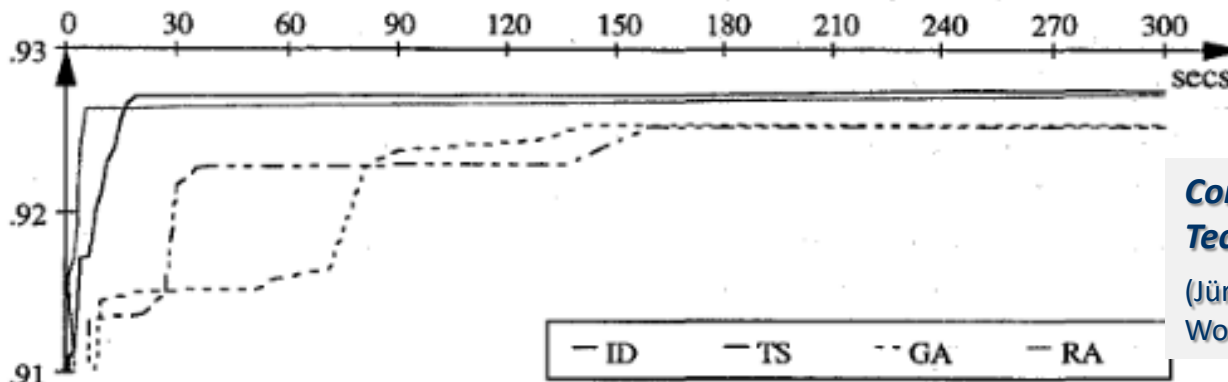


Fig. 12. Comparison of four algorithms applied to schedule A.



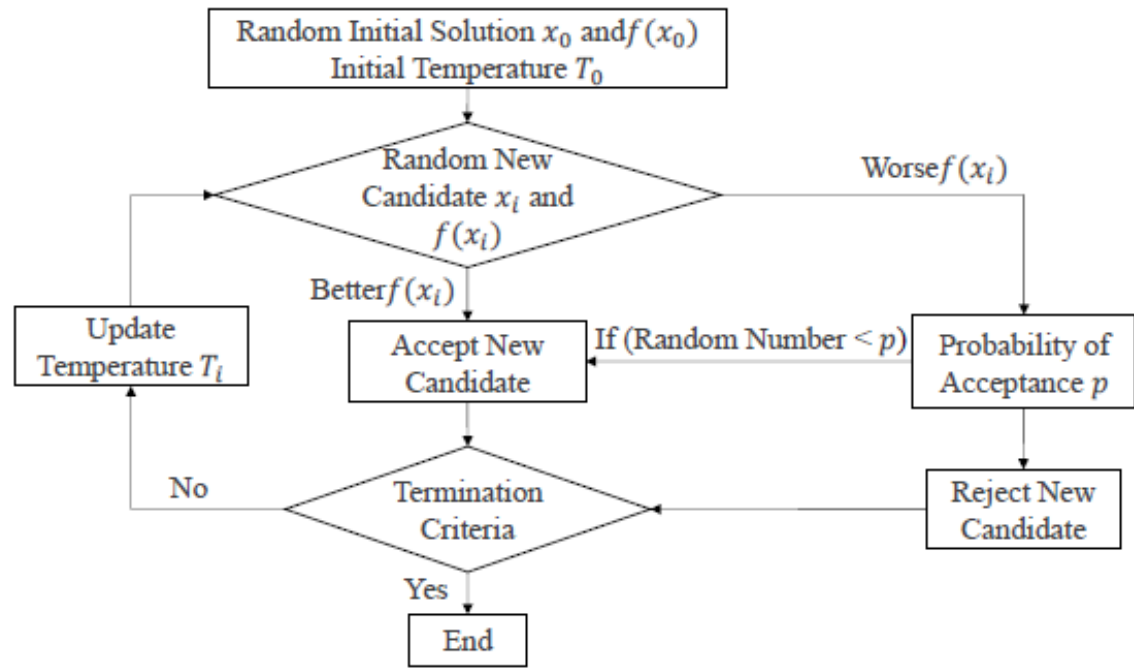
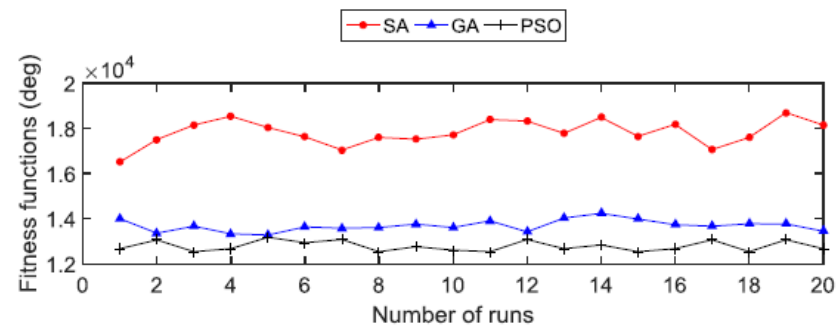
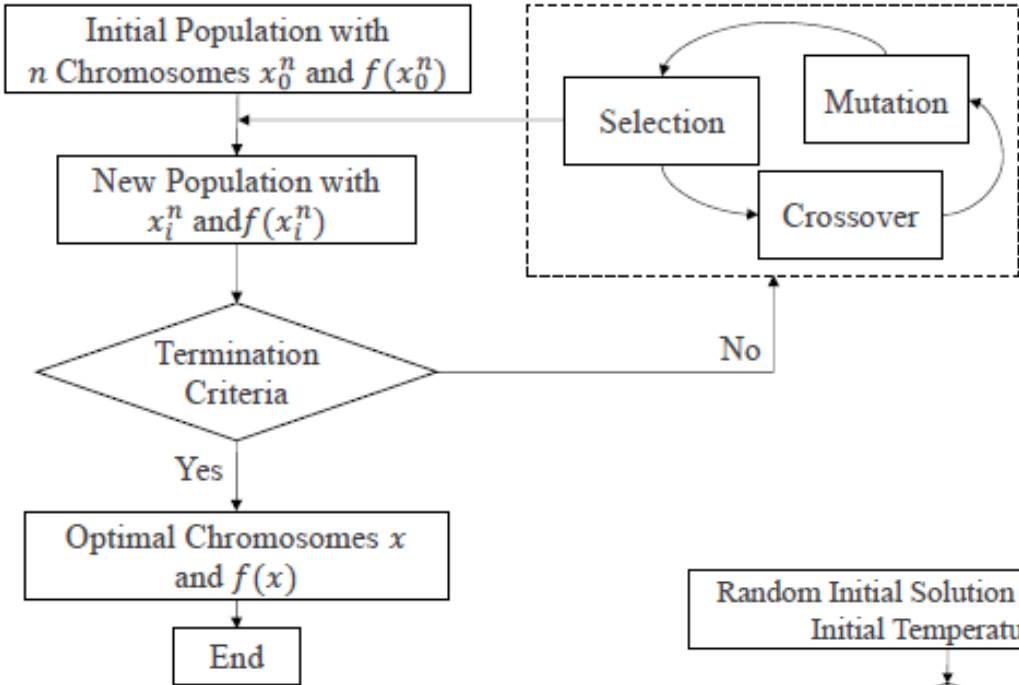
Comparison of Iterative Improvement Techniques for Schedule Optimization

(Jürgen Dorn, Mario Girsch, Günther Skele, Wolfgang Slany)

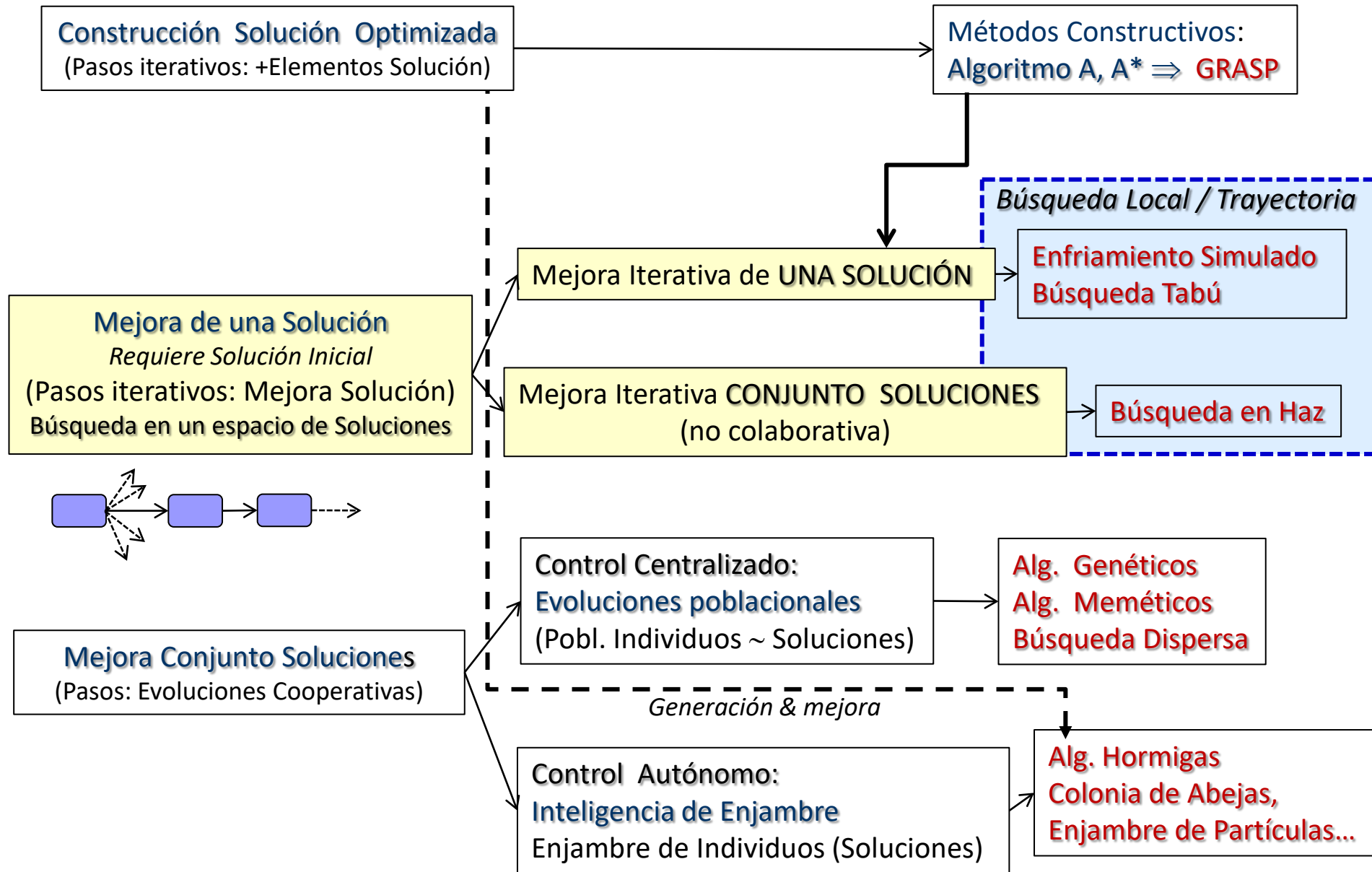
Fig. 13. Comparison of the four algorithms applied to schedule C.

Comparativa Algoritmo Genético vs Enfriamiento Simulado

A comparison of simulated annealing, genetic algorithm and particle swarm optimization in optimal first-order design of indoor tls networks. F.Jia, D. Lichti ISPRS'2017

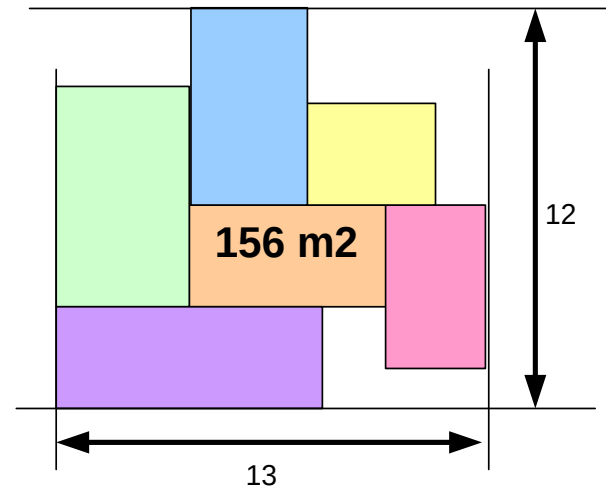
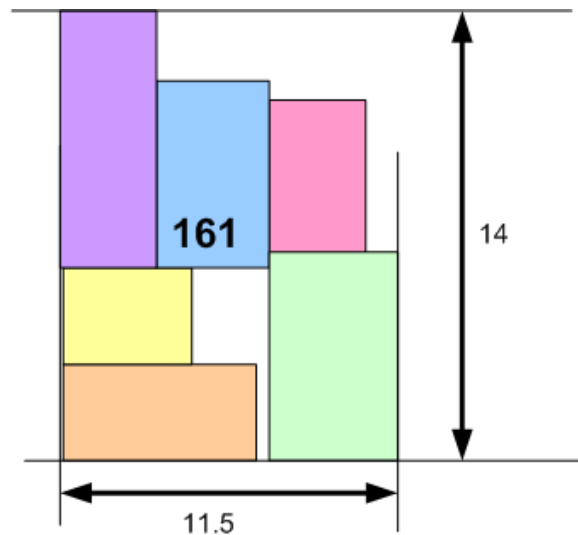


Métodos Metaheurísticos. Tipos Básicos.



EJEMPLO

Problema del Recubrimiento (con libre orientación)



Comparativa SA, TS, GA

Comparativa SA, TS, GA

Problema del Recubrimiento (con libre orientación)

Codificación de un individuo:

- Basada en una ‘estructura de cortes’.
- Asumiendo N bloques, Individuo se representa por un árbol binario, donde cada (N-1) nodo-corte representa un corte H/V y cada (N) nodo-bloque representa un bloque.
- Notación polaca para representar el árbol.
- Generación de individuos: Expresiones Polacas, con N-1 nodos-corte y N nodos bloque.

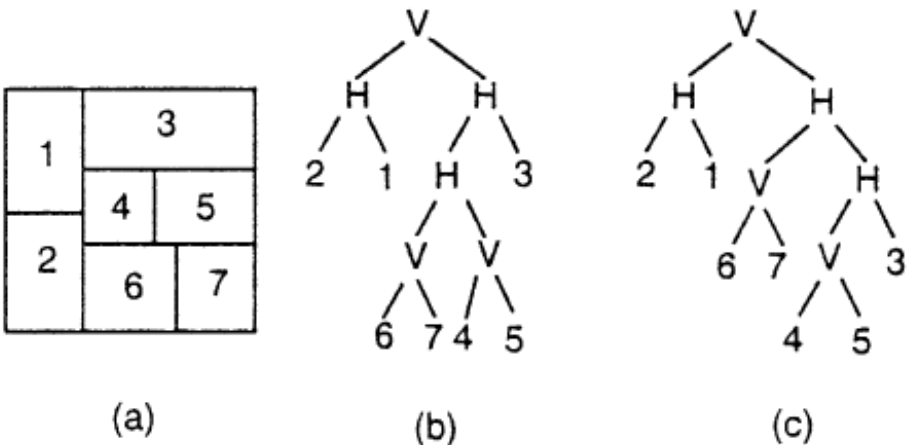
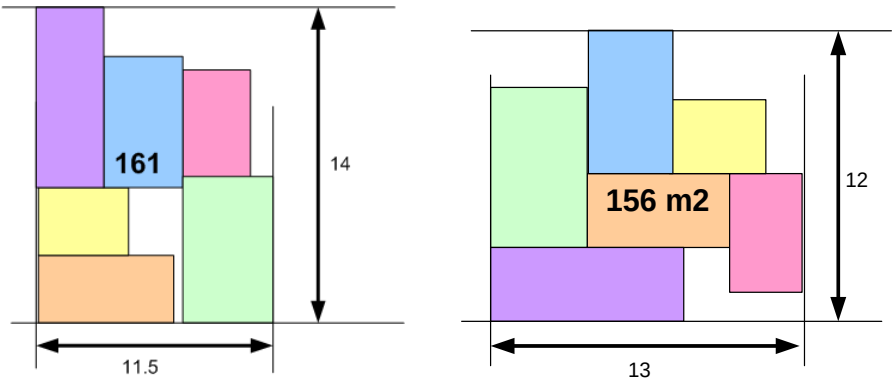
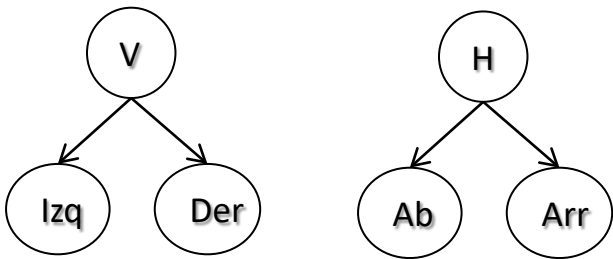


Fig. 3. (a) Slicing floorplan. (b) Slicing tree with Polish expression VH21HHV67V453. (c) Another possible slicing tree with Polish expression VH21HV67HV453.

Evolutionary algorithms, simulated annealing and tabu search:
a comparative study
H. Youssef et al. / Engineering Applications of Artificial Intelligence ,14

Diseño Algoritmo Genético

Poblacion Inicial: Individuos generados aleatoriamente. Expresiones Polacas, con N-1 gen-cortes y N gen-bloques.

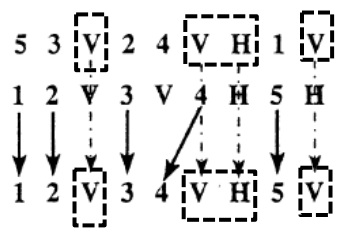
Ejemplo: VH21HHV67V453, VH12HV47HV653,

Selección: Ruleta

Cruce: Dos operadores, guiados para **heredar** la estructuras de “bloque” o “corte” de uno de los padres (es importante heredar propiedades del padre!)

a) Genes-bloque del Padre-1 se copian en Hijo-1. Luego, genes-corte del Padre2 se copian de izquierda a derecha para completar el cromosoma. El hijo **hereda** la **estructura de bloque** de uno de los padres.

P1= VH**21**HHV**67**V**453**
P2= **VH**12**HV**47**HV**653
HIJO-1= **VH21****HVH67****V453**

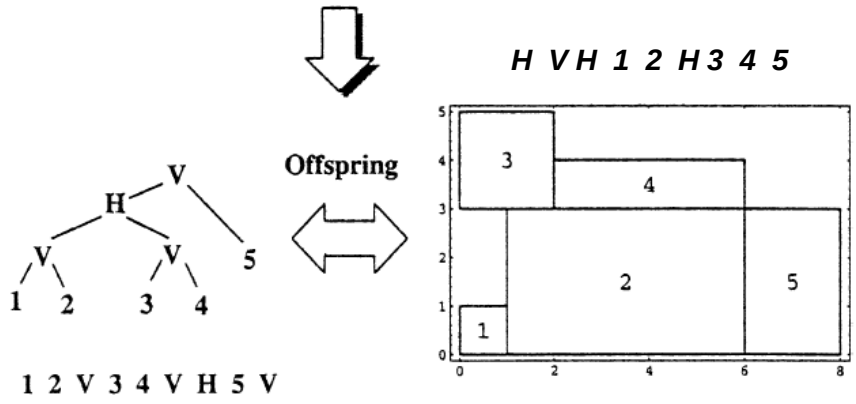


b) Genes-corte del Padre-1 se copian en Hijo-2.

Luego, genes-nodo del Padre1 se copian de izquierda a derecha para completar el cromosoma.

Hijo **hereda** la **estructura de corte**.

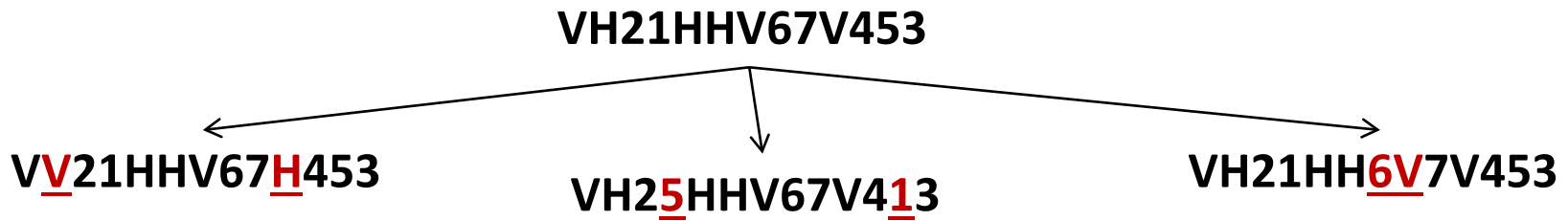
P1= **VH**21**HHV**67**V**453
P2= VH12HV47HV653
HIJO-1= **VH12HHV**47**V**653



Ejemplo Cruce-b: Hijo hereda estructura corte

Mutación:

- (1) intercambio de dos genes-corte seleccionados al azar
- (2) intercambio de dos genes-bloque seleccionados al azar
- (3) intercambiar un gen-bloque con un gen-corte adyacente, resultando una expresión válida.



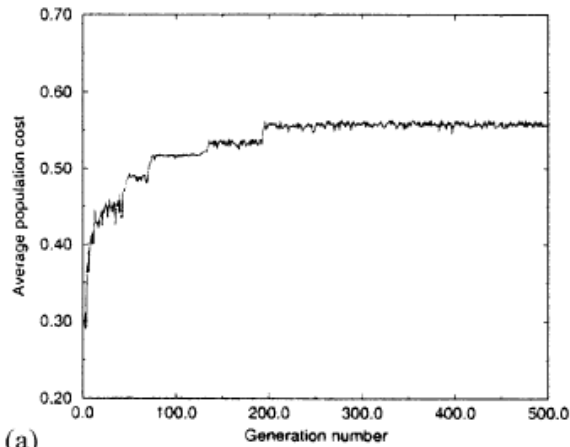
La mutación se aplica para la obtención de soluciones vecinas en SA y TS

ETAPAS DE DISEÑO

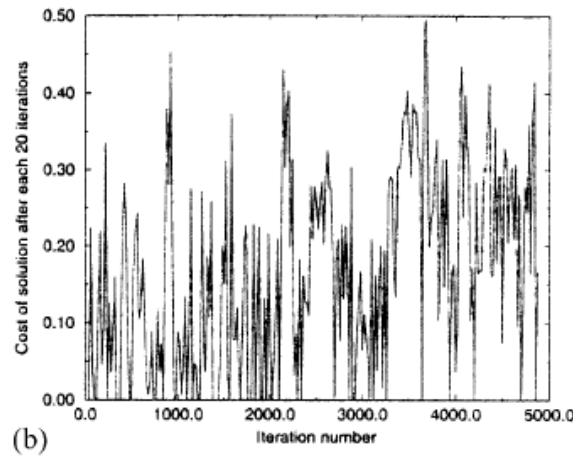
- 1) Obtener y analizar las soluciones (no óptimas) al problema. Obtener su **caracterización**.
- 2) Determinar el método para la **generación de soluciones no óptimas** (población inicial AG)
- 3) Definir la métrica de evaluación (**fitness**).
- 4) Diseño del individuo del AG (cromosoma):
- 5) Definir las operaciones de codificación / decodificación: INDIVIDUO ↔ SOLUCIÓN
- 6) **Determinar unas funciones de cruce/mutación adecuadas al diseño del individuo (generan nuevos individuos que corresponden a nuevas soluciones válidas)**

Evaluación

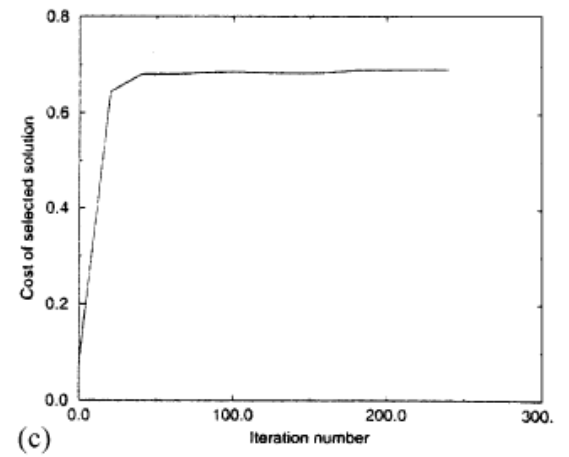
Genetic Algorithm



Simulated Annealing

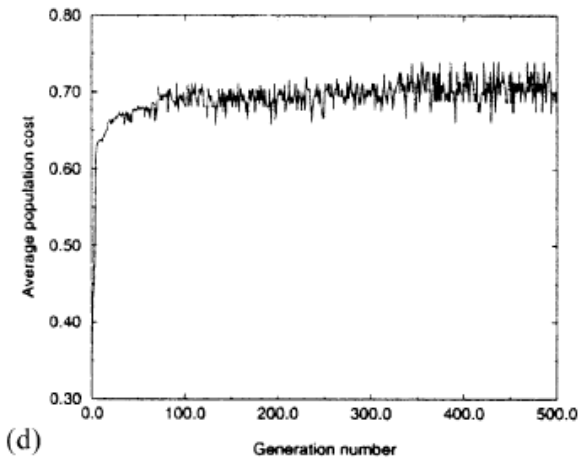


Tabu Search

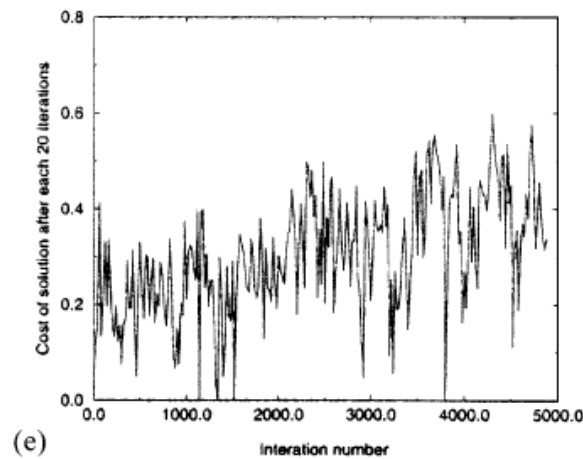


39 bloques

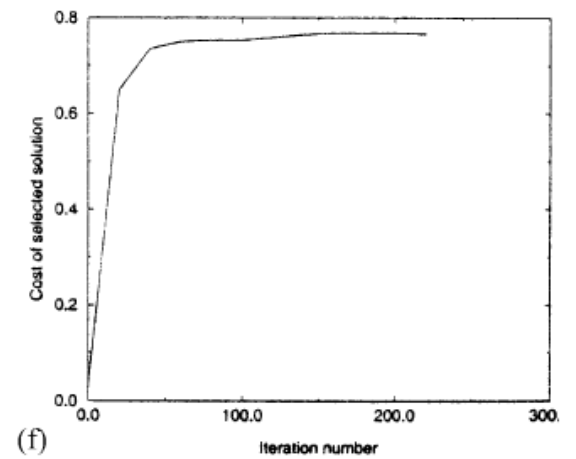
Genetic Algorithm



Simulated Annealing

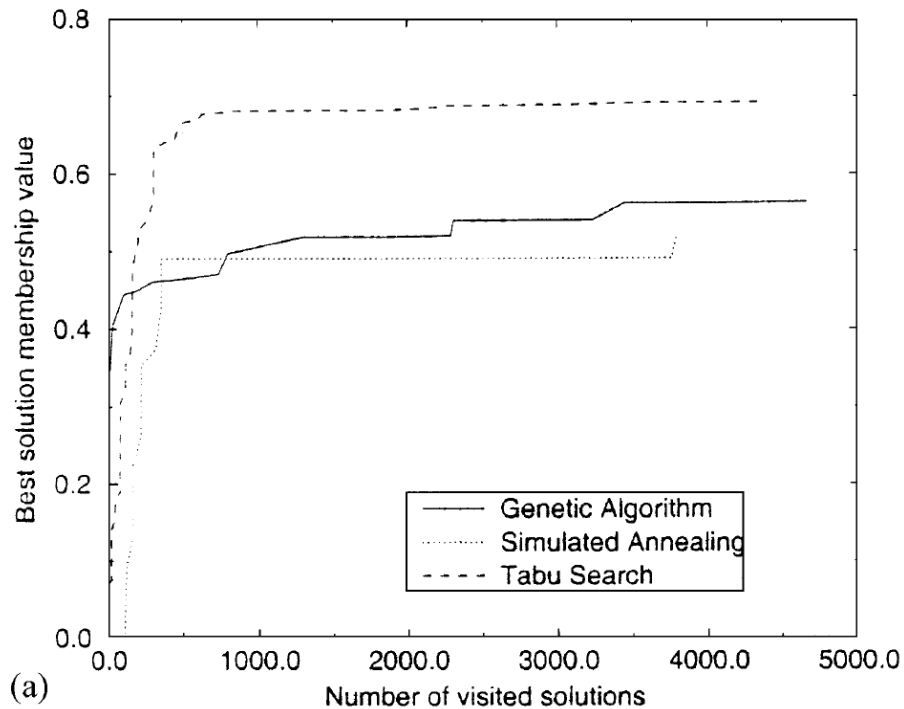


Tabu Search

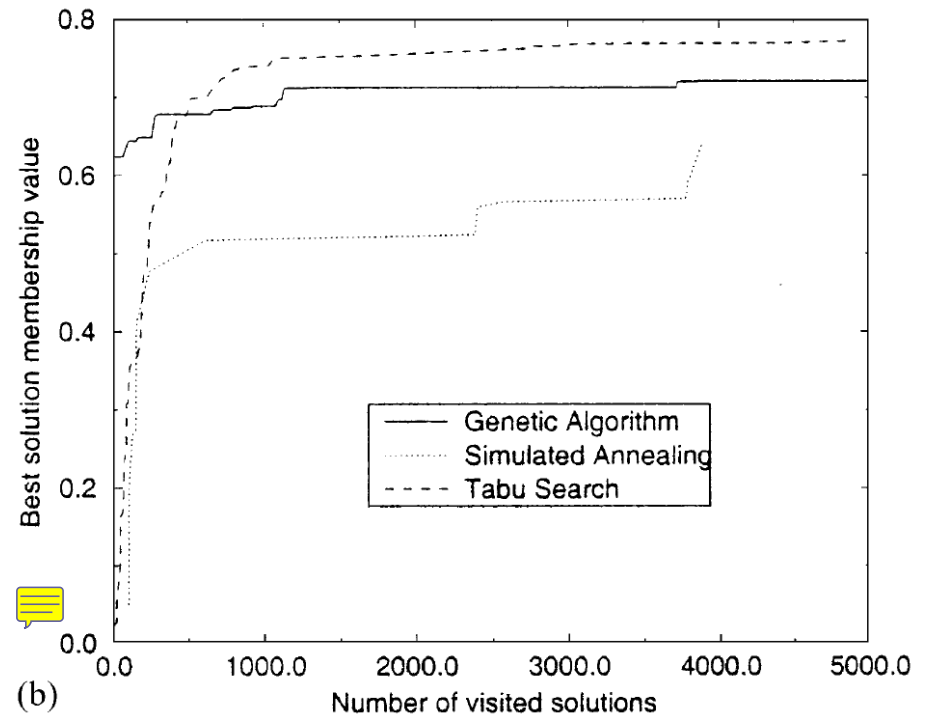


45 bloques

Evaluación



Benchmark: 39 bloques



Benchmark: 45 bloques